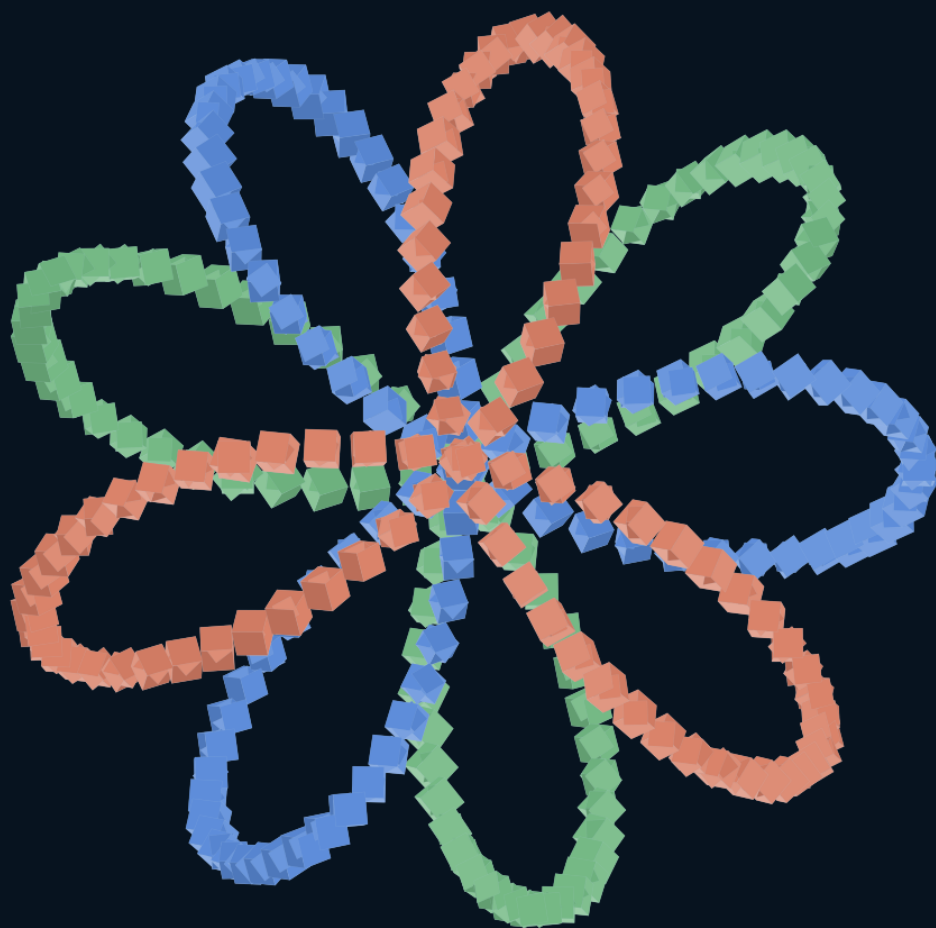




EDIÇÃO 1.0

SPATIALSEED

Manual do usuário, referência técnica e atlas procedural

Arquitetura - autoria - animação - jogo - PWA - obras reproduzíveis

Rogério Duarte

18 de agosto de 2026 - baseline 0054mv

Sumário

Esta edição integra fundamentos, manual do usuário, referência técnica, atlas procedural e critérios de evidência. As fontes editoriais e os exemplos estão anexados ao PDF.

PARTE I - O SPATIALSEED COMO SISTEMA	5
Uma definição operacional	6
O que existe, o que amadurece e o que ainda é horizonte	7
Cinco estados que não devem ser confundidos	8
A disciplina de uma mutação	9
Arquitetura em camadas	10
Por que o projeto usa comandos	11
O custo das abstrações	12
Como ler o restante do livro	13
 PARTE II - MANUAL DO USUÁRIO	 14
Sobre este manual	15
Iniciar o SpatialSeed	16
Confirmar o build carregado	17
Entender a tela	18
Primeiro percurso	20
Navegar e selecionar	21
Mover, girar e escalar	22
Duplicar, repetir e agrupar	23
Criar geometrias e luzes	24
Usar o plano ativo e ferramentas 2D	25
Caminhos, tubos, varreduras e distribuição	26
Editar uma malha	27
Aparência e Inspector	28
Salvar, abrir e recuperar	30
Importar e exportar STL	31
Console e linguagem afim	32
Animação efêmera	34
Câmera e modo somente cena	35
Viewers locais	36
Jogar com um personagem	37
Diagnóstico e testes	39

Solução de problemas	40
Atalhos e hábitos seguros	42
O que ainda não prometer	43

PARTE III - REFERÊNCIA TÉCNICA 44

Escopo e autoridade	45
Modelo de maturidade	46
Camadas de estado	47
Fluxo de uma mutação	48
Comandos, queries e eventos	49
Famílias principais de comandos	50
Fachada de autoria	51
Documento, projeto e arquivos	52
Geometria e recursos	53
Seleção e transformação	54
Planos, esboços e caminhos	55
Edição de malha	56
Runtime procedural	57
Tempo e animação	58
Runtime de jogo	59
PWA e build	61
Viewers e coordenação local	62
Perfis da aplicação	63
Testes e gates	64
Dívida e limites conhecidos	65
Mapa de pacotes	66
Critérios para documentação futura	67

PARTE IV - ATLAS PROCEDURAL 68

Modelo matemático	69
Onda integrada	70
Hélice ascendente	72
Roseta tricromática	74
Cidade policêntrica	76
Trindade orbital	78
Do atlas histórico às ferramentas atuais	80

PARTE V - EVIDÊNCIA, LIMITES E ROTEIRO 81

Como validar uma versão	82
-----------------------------------	----

Limites assumidos	83
Roteiro de maior retorno	84
Motivações para continuar	85

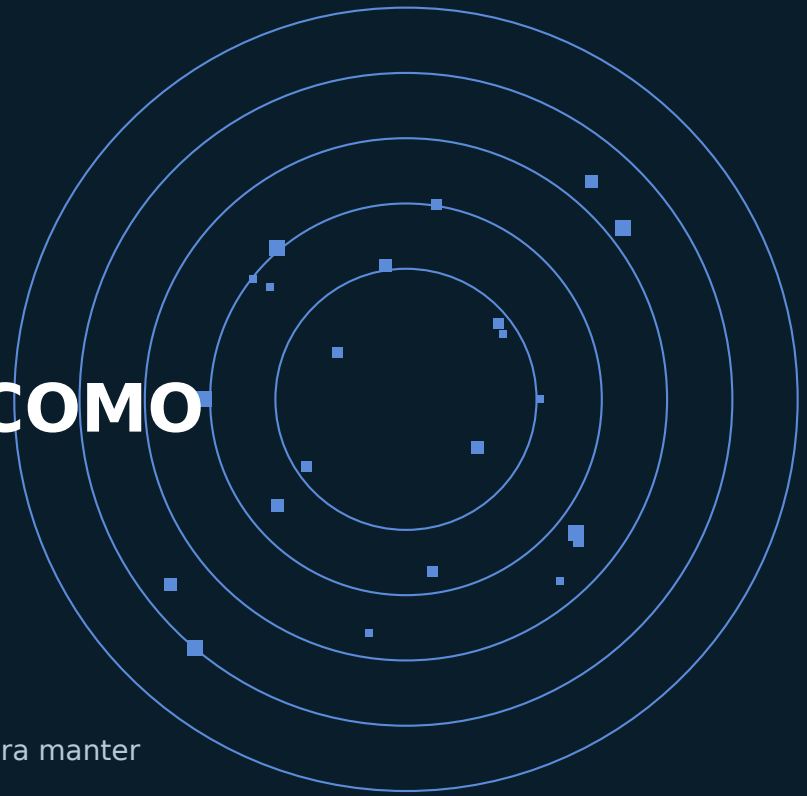
APÊNDICES **86**

Comandos de entrada	87
Símbolos matemáticos	88
Fontes anexadas	89
Nota editorial	90

FUNDAMENTOS

O SPATIALSEED COMO SISTEMA

Identidade, autoridade e estados distintos para manter interfaces diferentes sobre o mesmo mundo.



SpatialSeed é um ambiente para construir mundos digitais sem reduzir o mundo à tela que o mostra. A aplicação atual oferece edição 3D, programação procedural, animação e um loop jogável local. A questão que organiza o projeto é mais durável: como conservar identidade, autoria e capacidade de revisão quando o mesmo estado é operado por interfaces e projeções diferentes?

1

Uma definição operacional

Uma cena do SpatialSeed contém objetos lógicos, geometrias, aparências, hierarquias e propriedades. Uma sessão acrescenta seleção, pivô, câmera, previews, tempo e simulação. A distinção é essencial: o que descreve o mundo pode ser salvo; o que descreve uma forma momentânea de observá-lo ou manipulá-lo deve poder desaparecer sem corrompê-lo.

O mundo não é o `Object3D`, a aba, o painel ou o frame atual. O mundo é o estado que pode ser reconstruído pelos contratos aceitos.

Um botão e um comando textual pertencem à mesma aplicação quando traduzem a intenção para a mesma operação. Se cada superfície cria sua própria versão da lógica, o projeto deixa de ter uma autoridade única.

2

O que existe, o que amadurece e o que ainda é horizonte

Nível	Evidência exigida	Exemplos
Implementado	código e caminho observável	seleção, projetos, malha, STL, animação e jogo local
Testado	gate, suíte ou roteiro registrado	gates locais, testes do runtime e auditorias de marco
Em consolidação	contrato existente, ainda variável	colisão por malha, personagem GLB e ferramentas por caminho
Decidido	invariante registrada	preview não é commit; renderer não é autoridade
Pretendido	direção sem entrega completa	colaboração remota, física geral e modificadores vinculados

Essa tabela não é apenas uma convenção editorial. Ela impede que a arquitetura pretendida seja vendida como função presente e impede o erro inverso de ignorar capacidades que já existem porque ainda não alcançaram maturidade comercial.

3

Cinco estados que não devem ser confundidos

Estado canônico

Objetos, hierarquia, geometria, aparência e propriedades aceitas. É a parte que o projeto precisa reconstruir.

Estado editorial

Seleção, pivô, contexto de ferramenta, sessão de malha e histórico local. Ele organiza a autoria, mas não é uma propriedade permanente da cena.

Estado de viewer

Câmera de navegação, apresentação, painéis e renderização local. Dois viewers podem mostrar o mesmo mundo por câmeras diferentes.

Estado temporal

Relógios, domínios, operações e overlays de animação. Uma definição temporal pode ser compartilhada; matrizes por quadro permanecem locais.

Estado de simulação

Velocidade, contato, entrada, câmera de jogo e binding visual. Parar a sessão deve liberar demanda de frame e restaurar autoria.

4

A disciplina de uma mutação

Uma intenção persistente percorre comando, validação, resolução de alvos, serviço de domínio, sandbox e projeção. Uma intenção transitória pode parar no preview. Um programa pode produzir um plano e aguardar revisão.

```
interface -> comando -> validação -> sandbox -> projeção
programa  -> plano   -> revisão   -> comando -> sandbox
gesto     -> preview -> commit ou cancelamento
```

Essa disciplina permite substituir UI, renderer, índice espacial ou backend de script sem redefinir o significado da operação.

5

Arquitetura em camadas

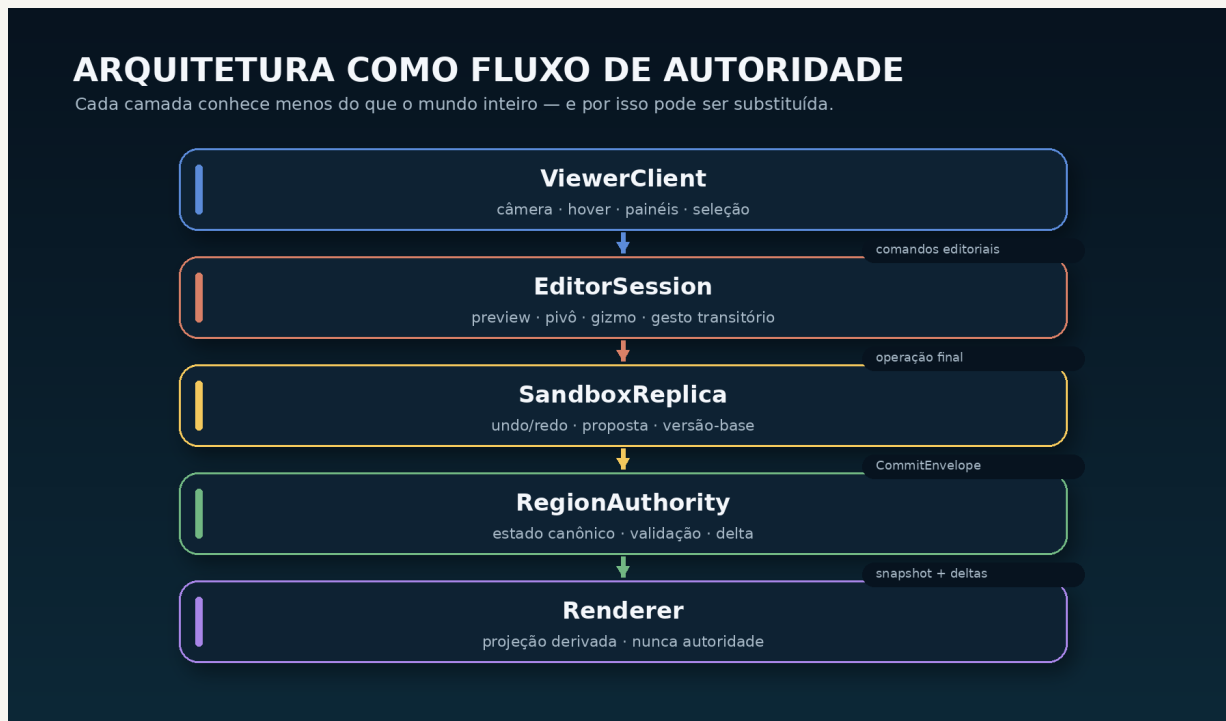


Diagrama editorial da separação entre estado lógico, edição, projeção e interfaces.

Região

Representa autoridade lógica e regras de aceitação. A arquitetura remota ainda é horizonte; o contrato local já impede que pixels decidam o estado.

Sandbox

Mantém réplica editorial, revisão, comandos aceitos e undo/redo.

Editor

Mantém seleção, pivô, ferramentas e sessões transitórias.

Viewer

Mantém câmera, apresentação, tempo e jogo.

Renderer

Materializa recursos, lotes, highlights e overlays. Ele pode ser otimizado sem apagar identidade lógica.

6

Por que o projeto usa comandos

Comandos tornam uma intenção nomeável, testável e reutilizável. `game.start`, `selection.translate` e `mesh.edit.commit` podem ser chamados por interfaces diferentes sem expor o objeto interno que implementa cada etapa.

Queries descrevem estado. Eventos comunicam fatos transitórios. Misturar os três conceitos produz dependências ocultas: uma leitura que muda o mundo, um evento que se torna autoridade, ou um botão que contorna o serviço canônico.

7

O custo das abstrações

Separar camadas tem custo fixo. O projeto não trata isso como motivo para remover contratos antes de medir. Benchmarks devem separar parser, validação, clone, publicação, renderer, DOM e GPU. A duração de `runtime test all` é teste de correção, não medida de desempenho.

O princípio econômico é simples: abstrações devem pagar seu custo por substituíbilidade, segurança, teste ou reuso real. Quando não pagam, a dívida deve ser reduzida por gates monotônicos.

Como ler o restante do livro

A Parte II é um Manual do Usuário orientado a tarefas. A Parte III é a Referência Técnica. Ambas são incluídas diretamente das fontes vivas, evitando que o PDF se torne uma documentação paralela. A Parte IV apresenta um atlas procedural e a Parte V encerra com evidência, limites e horizonte.

MANUAL

MANUAL DO USUÁRIO

Percursos concretos para instalar, criar, editar,
programar, animar, jogar, salvar e diagnosticar.

1

Sobre este manual

Este manual descreve tarefas concretas na aplicação web mantida em [apps/web/](#). Ele foi reescrito a partir do snapshot de 18 de agosto de 2026, derivado do build [20260818-0054mv](#). A árvore recebida passou pelos gates locais depois da correção do início automático do projeto demo e da inclusão da paleta.

O número exibido dentro do aplicativo continua sendo a autoridade para a versão realmente carregada. Uma fonte modificada no disco não prova que o navegador ou o service worker já a ativaram.

Três estados de maturidade

Estado	Como ler neste manual
Implementado	caminho de uso presente no snapshot atual
Em consolidação	funciona por uma superfície pública, mas ainda pode mudar em ergonomia, desempenho ou semântica
Pretendido	direção futura; não aparece como instrução operacional

Edição básica, arquivos de projeto, PWA, console, animação efêmera, edição de malha, STL e modo jogo local estão implementados. Colaboração remota, física geral, booleanas robustas e modificadores vinculados são arquitetura pretendida.

2

Iniciar o SpatialSeed

Android com Termux

Entre no checkout que deseja testar e inicie o servidor canônico:

```
cd ~/SpatialSeed
python3 tools/no_cache_server.py --port 8082
```

Mantenha essa sessão aberta. Em outra sessão do Termux:

```
termux-open-url 'https://127.0.0.1:8082/apps/web/'
```

O servidor usa HTTPS por padrão. Na primeira execução, instale no Android a autoridade certificadora local indicada no terminal. Sem uma origem confiável, recursos como service worker, instalação da PWA e alguns acessos a arquivo podem se comportar de modo diferente.

Para testar a partir de outro aparelho na mesma rede:

```
python3 tools/no_cache_server.py --port 8082 --network
```

Use uma das URLs impressas pelo servidor. Não exponha essa origem a redes não confiáveis.

Desktop

O mesmo servidor funciona num checkout desktop:

```
cd SpatialSeed
python3 tools/no_cache_server.py --port 8082
```

Abra <https://127.0.0.1:8082/apps/web/>. O projeto não exige `npm install` nem uma etapa de build para executar a aplicação. Node.js é usado por parte da validação de desenvolvimento.

URLs úteis

URL ou parâmetro	Resultado
/apps/web/	abre o projeto de demonstração padrão
/apps/web/?project=new	inicia um projeto vazio
/apps/web/?application=diagnostics	carrega o perfil diagnóstico
/apps/web/?viewer=join	participa de uma sessão local quando o fluxo correspondente fornece o sandbox

O projeto demo é definido por `apps/web/assets/demo/default-game.manifest.json`. O manifesto escolhe arquivo, personagem, modo de lançamento e controles; o bootstrap não depende do nome ou do índice visual do personagem. Antes de iniciar o jogo, a aplicação conclui a recuperação local. Um projeto recuperado permanece no modo de autoria; o demo só entra em jogo quando não há projeto persistente ou quando o rascunho é descartado.

3

Confirmar o build carregado

O painel **Status e comandos** mostra um rótulo como `v0.1.0 - build 20260818-0054mv`. Esse é o build publicado lido com `no-store`. O título do campo contém detalhes adicionais:

- build publicado;
- build do service worker controlador;
- worker ativo;
- worker aguardando;
- eventual falha de atualização.

Atualizar agora fica disponível quando o controlador não coincide com o build publicado ou quando a versão publicada está sendo instalada. Ao concluir, o aplicativo ativa o worker correspondente e recarrega.

Se o rótulo mostrar um build antigo no cache:

- 1 pressione **Atualizar agora**;
- 2 se aparecer, use **Reparar atualização**;
- 3 feche todas as abas e a janela PWA instalada;
- 4 reabra exatamente a origem do checkout em teste;
- 5 confirme novamente build publicado e controlador.

Uma mudança no código que conserva o mesmo identificador de build é ambígua para o cache. Releases de código devem receber um build novo e regenerar o manifesto de precache.

4

Entender a tela

Viewer

O viewer ocupa a área central. Ele projeta a cena, recebe navegação, seleção, gestos de ferramentas e gizmos. O renderer não é a fonte autoritativa do projeto; ele materializa o estado lógico e overlays transitórios.

Barra principal

A barra reúne:

- navegar, selecionar, mover, girar e escalar;
- operações de seleção;
- alternância mundo/local;
- criação rápida, undo, redo e revisão;
- Estrutura, Diagnóstico, Desenvolvedor e Console;
- Câmera, Render, viewers, Inspector e edição;
- Criar, Explorar, Animação e Jogar;
- modo somente cena;
- Salvar, Abrir, Novo, STL, personagem GLB e procedimentos.

HUD Editar

O botão com lápis abre o HUD contextual. Ele concentra:

- assunto: objeto, vértice, aresta ou face;
- ferramenta: navegar, selecionar, mover, girar ou escalar;
- operação de seleção: substituir, adicionar, remover ou alternar;
- frame: mundo, local, viewer ou plano personalizado;
- eixos, snap, proporcionalidade e travas;
- plano ativo de autoria;
- ferramentas 2D, medição, caminhos e operações topológicas;
- aplicar ou cancelar a sessão de malha.

O botão ? ativa ajuda dos ícones. O layout pode ser redimensionado e personalizado; preferências de apresentação pertencem ao navegador, não ao arquivo do projeto.

Painéis

Painéis como Inspector, Câmera, Render, Console e Editar podem coexistir. Em telas pequenas, feche os que não estiver usando. A posição de um painel é estado de interface e não aparece no undo.

Paleta de comandos

Use **Comandos** na barra ou **Ctrl+P** (**Cmd+P** no macOS). Digite parte do nome, ID, categoria ou atalho. Ações de interface são executadas diretamente. Um comando do runtime que possa exigir parâmetros é aberto no Console, sem ser executado silenciosamente com argumentos ausentes. Use as setas para navegar, **Enter** para escolher e **Esc** para fechar.

5

Primeiro percurso

- 1 Abra a URL comum. O projeto demo deve carregar e entrar em jogo.
- 2 Ande, corra e pule; depois pressione **Sair do jogo**.
- 3 Escolha **Selecionar** e toque ou clique num objeto.
- 4 Escolha **Mover** e arraste uma alça do gizmo.
- 5 Pressione **Desfazer local** e **Refazer local**.
- 6 Abra **Criar**, escolha uma geometria e posicione-a.
- 7 Abra **Inspector** e altere a cor.
- 8 Pressione **Salvar** e guarde o arquivo `.spatialseed`.

Esse percurso verifica carregamento, projeção, seleção, mutação, histórico e transporte de arquivo sem exigir o console.

6

Navegar e selecionar

Navegação

Use a ferramenta **Navegar** quando quiser mover a câmera sem iniciar uma operação editorial. Mouse e toque são mediados pelo viewer. Durante ferramentas de desenho, pan e pinch continuam disponíveis conforme a arbitragem de ponteiros; gestos com mais dedos podem ser reservados à órbita.

Seleção direta

Com **Selecionar**, toque ou clique num objeto. O painel de status informa a quantidade e o membro ativo. A seleção pertence ao editor e não é salva como parte do mundo.

Operações de seleção

Operação	Efeito
Substituir	troca a seleção pelo resultado do gesto
Adicionar	inclui o resultado
Remover	exclui o resultado da seleção atual
Alternar	inclui membros ausentes e remove membros presentes

Use seleção única para trabalho direto e múltipla para operações em lote.

Gestos de área

O HUD e a barra oferecem:

- retângulo;
- pincel;
- laço;
- borracha.

O gesto captura uma intenção geométrica e resolve o conjunto de objetos de modo atômico. O índice espacial reduz o custo de consultar a cena inteira em cada movimento.

Grupos

Um grupo selecionado pode ser tratado como raiz rígida. Algumas operações oferecem escopo direto ou expansão explícita em objetos renderizáveis. Não presuma que selecionar um grupo equivale a editar todos os descendentes.

7

Mover, girar e escalar

Escolher a ferramenta

Use a barra ou o HUD para ativar mover, girar ou escalar. O gizmo mostra alças por eixo e, quando aplicável, por plano ou combinação uniforme.

Frames

Frame	Interpretação
Mundo	eixos globais da cena
Local	eixos do objeto ou contexto ativo
Viewer	eixos derivados da vista atual
Plano personalizado	base do plano ativo de autoria

O frame orienta a transformação. Ele não é o mesmo que plano de desenho nem trava de visualização 2D.

Pivô

Políticas comuns:

- mediana;
- centro dos limites;
- objeto ativo;
- posição personalizada.

Editar pivô muda o ponto de manipulação, não a geometria do objeto. Um pivô personalizado pode ser movido pelo HUD e usado em rotação, escala e repetição.

Snap e travas

Configure passos de grade, ângulo e escala no HUD. Travas podem limitar a operação a eixos, plano ou ponto. Verifique o indicador do contexto antes de arrastar; um gizmo aparentemente parado pode estar corretamente bloqueado por uma restrição ativa.

Preview e commit

Durante o arrasto, o renderer mostra um preview. Ao concluir, no máximo um comando persistente entra no sandbox e no histórico. Cancelar descarta o preview sem mutar o documento.

8

Duplicar, repetir e agrupar

Duplicação simples

Selecione um objeto ou grupo e pressione **Duplicar**. A cópia inicia uma sequência de repetição. Transformações confirmadas depois da duplicação compõem uma matriz delta.

Repetir

Pressione **Repetir** para aplicar novamente a transformação composta. No console:

```
duplicate  
move 2 0 0  
rotate 0 15 0  
repeat count 12
```

O lote é preparado antes da publicação e entra como uma operação editorial.

Agrupar e desagrupar

Agrupar cria uma unidade lógica com transformações locais. **Desagrupar** remove um nível sem deslocar os filhos no espaço mundial. Grupos aninhados são permitidos, mas recursos e ocorrências podem usar resolução hierárquica; teste o preview quando combinar grupos, instâncias e animação.

Criar geometrias e luzes

Painel Criar

Abra **Criar**, escolha uma família, ajuste os parâmetros e use o posicionamento no viewer. O catálogo é fornecido pelo [GeometryRegistry](#); a lista exata deve ser consultada na versão carregada.

Famílias comuns incluem caixa, esfera, cilindro, cone, plano, polígono, cápsula, círculo, anel, toro, nó toroidal, poliedros, revolução, tubo, forma, extrusão e buffer.

Colocação

Uma geometria pode ser definida em:

- planos mundiais XY, XZ ou YZ;
- plano ativo;
- base formada por origem, normal e tangente;
- três pontos;
- objeto, face ou superfície capturada.

O preview de posicionamento é local. Confirme para criar o objeto ou cancele para não alterar o projeto.

Criação pelo console

```
create box size 2 1 3 color #4488ff
create sphere radius 0.5 origin 2 1 0
create polygon sides 6 radius 2 plane xz
```

Consulte [help create](#) para parâmetros efetivos.

Luzes

O workspace Editar oferece criação de luz com valores memorizados. Luzes são objetos persistentes: podem ser selecionadas, transformadas e salvas. As preferências gerais de renderização do viewer, por outro lado, permanecem locais.

Usar o plano ativo e ferramentas 2D

Plano ativo

O SpatialSeed apresenta um único plano ativo de autoria. Ele alimenta criação, formas 2D, caminhos, medição e gestos de malha. Fontes possíveis incluem:

- viewer;
- XY, XZ ou YZ mundial;
- objeto;
- face;
- três pontos;
- superfície;
- helper personalizado.

Definir e travar captura a fonte atual. **Editar helper** permite mover ou girar o plano. **Liberar plano ativo** retorna ao fallback.

Formas 2D

O HUD contém ponto, segmento, polilinha, retângulo, círculo, arco e polígono. Polilinhas possuem ações de concluir, remover último ponto e cancelar.

Essas formas preservam um esboço semântico independente da malha usada para exibí-las. Um círculo fechado pode servir posteriormente como perfil.

Medição

Régua e transferidor usam o plano ativo. Medições pertencem ao viewer e servem de referência durante a autoria; não devem ser confundidas com cotas persistentes de CAD, ainda pretendidas.

11

Caminhos, tubos, varreduras e distribuição

Referências espaciais

Objetos podem oferecer papéis de caminho, perfil ou pontos. A resolução produz snapshots tipados; nesta geração, alterações posteriores na fonte não regeneram automaticamente todos os resultados como um modificador vinculado.

Tubo

Selecione ou forneça um caminho e crie um tubo. Ajuste raio, segmentos e fechamento. A linha central de uma geometria compatível pode ser reutilizada.

Varredura

Uma varredura recebe perfil e caminho em slots distintos. O frame usa transporte paralelo para reduzir torção artificial. Verifique orientação, tampas e twist.

Distribuição ao longo de caminho

Uma seleção pode ser repetida ao longo de um caminho com alinhamento e twist. No modo desenhado, o gesto cria instâncias progressivamente conforme a distância acumulada supera o espaçamento configurado.

Extrusão por caminho

Na edição de malha, `mesh.extrude` aceita:

- normal e distância;
- reta entre início e fim do arrasto;
- polilinha desenhada;
- caminho explícito fornecido por comando.

O preview topológico sempre parte do snapshot-base do gesto. Movimentos de ponteiro não acumulam extrusões independentes.

Editar uma malha

Entrar na sessão

Selecione um objeto compatível e pressione **Editar** ou **Editar seleção**. A sessão isola uma malha de trabalho e possui histórico interno. O documento só é substituído quando você pressiona **Aplicar**.

Modos de componente

Escolha vértices, arestas ou faces. As operações de seleção incluem todos, nenhum, inverter, expandir, contrair, conectados e contorno.

Transformação de componentes

Mover, girar e escalar usam o mesmo contexto de frames, eixos, snap e proporcionalidade do workspace. A influência proporcional afeta vizinhos durante o preview conforme as opções da sessão.

Operações disponíveis

Conforme o modo e a seleção, o HUD expõe:

- criar vértice, aresta ou face;
- preencher contorno;
- soldar;
- extrudar;
- inset;
- dividir ou colapsar aresta;
- inverter diagonal;
- ponte entre contornos compatíveis;
- subdividir;
- inverter ou recalcular normais;
- criar caminho da seleção;
- limpeza da malha.

Nem toda combinação topológica é válida. Uma operação deve falhar sem publicar resultado parcial quando seus pré-requisitos não forem satisfeitos.

Undo interno e undo do projeto

Use o undo interno enquanto a sessão está aberta. **Aplicar** transforma o resultado final em uma única alteração editorial; depois disso, o undo global pode restaurar a geometria anterior. **Cancelar** descarta a sessão.

Limites

Knife, bisect, atributos por canto, UV avançado, booleanas robustas e reparo geral de auto-interseção ainda não formam um conjunto estável completo.

Aparência e Inspector

Edição literal

Selecione objetos, abra **Inspector**, altere uma propriedade e aplique. O Inspector distingue valor uniforme, misto e propriedade não suportada.

Geometrias paramétricas preservadas, como extrusão, tubo, esfera e lathe, também publicam no Inspector os parâmetros declarados pelo próprio provider. Por exemplo, uma extrusão expõe profundidade, passos e bisel. Aplicar vários campos produz uma única operação de undo; salvar e reabrir preserva os valores.

Propriedades comuns:

- nome;
- posição, rotação e escala;
- cor, opacidade e transparência;
- textura, repetição, deslocamento, rotação e wrapping;
- cor por instância;
- parâmetros específicos de geometria ou luz;
- JSON estruturado para contornos, pontos, furos e outros parâmetros compostos.

Aplicação procedural em lote

Expressões podem calcular um valor por alvo. Exemplo:

```
property batch instance.color "hsl(300 * u,0.8,0.55)" scope=renderables
property batch transform.position "x; y + 2 * sin(tau * u); z"
property batch transform.scale "1; 0.5 + u; 1"
```

O programa é compilado antes da mutação. Se qualquer alvo falhar, o lote inteiro é rejeitado.

Camadas de cor

O Inspector distingue **Cor-base do material**, **Fonte da cor**, **Cor uniforme**, **Matiz final**, **Cor própria da instância** e **Cor efetiva**. A cor-base pertence ao material compartilhável. A fonte escolhe entre herdar essa base, usar uma cor uniforme ou obter cor por instância. O matiz é multiplicado ao resultado; a cor efetiva é somente leitura e mostra a composição vigente.

A cor exibida no HUD pode atuar sobre a ferramenta, a seleção ou ambas, conforme o campo **Aplicar em**. A cor da ferramenta vale para objetos criados em seguida; ela não é uma quarta cor persistente do objeto.

Copiar e colar propriedades

No Inspector, escolha um preset e pressione **Copiar**. **Rotação e escala** é o preset inicial e não move o destino. **Posição absoluta** fica separado e avisa que os objetos podem ficar coincidentes. Material e cor-base, textura, regra e matiz de cor, cor própria da instância e aparência completa também são presets distintos.

Depois de seleccionar o destino, abra **Confirmar propiedades**. Cada linha mostra o valor da origem, o valor atual do destino e uma caixa de seleção. Propriedades incompatíveis, não editáveis em lote ou já iguais ficam desativadas. Marque apenas o que deseja substituir e pressione **Aplicar propiedades marcadas**. A aplicação confirmada produz uma única entrada de undo/redo.

No console, use `property presets`, `property copy transform`, `property clipboard` e `property paste all`. Para restringir a colagem, informe IDs, por exemplo `property paste transform.rotationDeg transform.scale`. A cópia é local à sessão e não depende de permissão do clipboard do sistema.

Salvar, abrir e recuperar

Salvar

Pressione **Salvar**. Quando a File System Access API estiver disponível, o navegador pode permitir escolher e reutilizar um arquivo. Em ambientes móveis, o fallback produz download.

O arquivo `.spatialseed` representa o projeto lógico. Câmera de navegação, seleção, painéis, animação efêmera e estado físico do jogo não são gravados como quadros.

Abrir

Pressione **Abrir** e escolha um projeto. A substituição integral cancela sessões transitórias antes de instalar a nova cena. Uma falha operacional deve ser apresentada de forma recuperável sempre que possível.

Novo

Novo cria uma cena independente na aba atual. O menu de viewers também pode abrir um novo projeto em outra aba.

Recuperação local

O navegador mantém checkpoint e journal de comandos confirmados. Ao detectar rascunho recuperável, a aplicação oferece:

- Continuar;
- Exportar cópia;
- Descartar.

Recuperação local não substitui salvar um arquivo. Ela também não deve persistir previews, seleção, câmera ou jogo.

Ao recarregar depois de editar, o diálogo de recuperação deve permanecer visível e operável. **Continuar** restaura o projeto e abre a interface de autoria, sem reativar o modo jogo. **Descartar** remove o rascunho daquela origem e permite que o projeto demo volte a ser o fallback. Não é necessário apagar cache ou dados do site para sair desse fluxo.

15

Importar e exportar STL

Importar

Use **Importar STL**. O codec aceita STL ASCII ou binário. Por padrão, vértices coincidentes são reunidos com tolerância proporcional à diagonal da geometria. STL não informa unidade; ajuste a escala explicitamente quando necessário.

O resultado se torna um descritor `buffer` editável.

Exportar

Selecione um ou mais objetos e use **Exportar STL**. A exportação materializa a superfície final, aplica transformações mundiais e agrega os triângulos.

STL não preserva:

- materiais;
- UV;
- hierarquia;
- animações;
- identidade topológica semântica.

Use STL como transporte de superfície, não como formato mestre do projeto.

16

Console e linguagem afim

Começar pela ajuda

```
help
help create
help camera
help tool
procedure help
```

A ajuda da versão carregada tem precedência sobre listas estáticas.

Comandos editoriais

```
create box size 1 2 3
position 0 1 0
move 2 0 0
rotate 0 30 0
scale 1 2 1
duplicate
repeat
group Estrutura
ungroup
delete
undo
redo
```

Séries afins

Expressões conhecem *i*, *u*, *count*, posição e escala correntes, constantes e funções matemáticas:

```
create sphere radius 0.3 count 40 move "4 * cos(i * pi / 20)" 0 "4 * sin(i * pi / 20)"
duplicate count 24 move "3 * cos(i * pi / 12)" 0 "3 * sin(i * pi / 12)"
```

i começa em 1. *u* percorre o intervalo normalizado de 0 a 1. Rotações editoriais usam graus; trigonometria comum usa radianos, salvo funções com sufixo *d*.

Planejamento por JavaScript

`calc` e `program` executam JavaScript síncrono num Worker isolado por SES. O programa não recebe DOM, rede ou acesso irrestrito à cena. Quando autorizado, `spatial.create` produz intenções num plano pendente.

```
calc sqrt(3 ** 2 + 4 ** 2)
program return spatial.create("box", {size:[2,8,2],position:[0,4,0]})
plan status
plan commit
```

`plan commit` valida revisão e lote antes da publicação. `plan discard` descarta sem efeito.

Procedimentos

```
procedure define tower ({height=8}={}) =>  
  -> spatial.create("box",{size:[2,height,2],position:[0,height/2,0]})  
procedure run tower {"height":12}  
plan commit  
procedure export
```

Importar uma biblioteca armazena fonte; não executa automaticamente.

17

Animação efêmera

Presets

Selecione objetos e abra **Animação** ou use:

```
animate spin speed=45 axis=y  
animate orbit radius=4 speed=30 axis=y  
animate wave amplitude=1 frequency=0.5 mode=objects  
animate rainbow speed=60 saturation=0.8 mode=objects  
animate status  
animate stop
```

`mode=selection` trata raízes selecionadas como unidades rígidas. `mode=objects` expande grupos em objetos renderizáveis.

Faixas

O painel permite capturar seleções diferentes e associar presets distintos. Todas as faixas são avaliadas numa sobreposição temporal comum.

Persistência

Animação de preview não altera documento, undo ou arquivo. Parar deve restaurar a projeção canônica. Uma edição persistente pode interromper ou reconciliar a sessão conforme o contrato da operação.

Runtime legado

`AnimationRuntime` de passo fixo permanece para regressões históricas. Novas integrações devem usar `TemporalAnimationRuntime`. Essa distinção é técnica e não muda o fluxo comum do painel.

18

Câmera e modo somente cena

Câmera de navegação

O painel **Câmera** controla posição, alvo derivado, projeção, near, far, enquadramento, órbita e reset. Essa câmera pertence ao viewer e não ao documento.

Objetos câmera

Objetos câmera são persistentes. Você pode:

- criar a partir da vista atual;
- ativar no viewer;
- selecionar e transformar;
- capturar novamente a vista;
- definir câmera padrão do documento;
- desativar e retornar à navegação livre.

Cada viewer escolhe localmente qual câmera usa.

Modo somente cena

Esse modo oculta superfícies de autoria para apresentar a cena. Ele não é o mesmo que modo jogo. Use o hotspot ou a ação de saída para restaurar a interface.

Viewers locais

O painel de projetos/viewers descobre sessões da mesma origem. Uma aba pode abrir outra conectada a um sandbox escolhido. Cada viewer conserva câmera, seleção e painéis próprios; comandos persistentes são coordenados por revisão.

Se uma intenção foi preparada sobre revisão antiga, ela é rejeitada e o viewer recebe o estado atual. Essa coordenação usa [BroadcastChannel](#); não é colaboração remota nem multiusuário pela internet.

Também é possível abrir novo projeto ou arquivo em outra aba sem anexá-lo à sessão atual.

Jogar com um personagem

Projeto demo

Na URL comum, o manifesto demo seleciona o proxy físico. Depois de resolver a recuperação persistente, a aplicação inicia `game.start` somente se nenhum projeto anterior tiver sido restaurado. Se o início automático falhar, o editor deve permanecer recuperável e registrar o erro no console.

Iniciar manualmente

- 1 selecione um objeto renderizável;
- 2 pressione **Jogar** ou use a ação configurada;
- 3 use **WASD** ou setas, **Shift** e **Espaço**;
- 4 arraste o viewer para controlar a câmera;
- 5 pressione **Sair do jogo** ou **Esc**.

Em toque, arraste o botão central dentro do círculo direcional. Ângulo e distância ao centro controlam, respectivamente, direção e intensidade; isso também orienta a malha visual em rampas. A OBB física pode conservar por um instante a última orientação livre quando girar o volume causaria interseção com o piso ou uma parede. permite diagonais e movimento suave. Soltar recentraliza e interrompe o movimento. Corrida e Pular conservam ponteiros próprios e podem ser combinados com o círculo.

Proxy físico e visual GLB

O objeto selecionado é a autoridade física. Seu frame define centro, meia extensão e yaw. O visual GLB é projetado por uma raiz transitória independente, com escala e alinhamento próprios. Aumentar o proxy deve aumentar o collider, sem necessariamente aumentar a raposa ou outro rig.

As malhas do visual GLB emitem e recebem sombras quando sombras estão habilitadas nas opções do viewer. O piso de sombra continua pertencendo ao renderer, e não ao asset do personagem.

Use **Carregar personagem GLB** para associar um asset. Clips são vinculados a `idle`, `walk`, `run`, `jump`, `fall` e `land` quando os nomes permitem. O painel **Visual do personagem** fica acessível no HUD de jogo.

Movimento e câmera

O padrão é movimento relativo à câmera. Frente/trás seguem a frente horizontal da vista; esquerda/direita seguem a lateral. O modo mundial permanece configurável. A câmera de terceira pessoa mantém órbita livre e recua diante de obstáculos consultados no mundo de colisão.

Colisão

O mundo estático usa broad phase espacial. A narrow phase pode representar:

- caixa local;
- esfera analítica válida;
- malha triangular final.

O personagem usa OBB orientada pelo próprio yaw, enquanto a AABB conservadora fica restrita à busca ampla. A cinemática inclui gravidade, aceleração, atrito, pulo, tolerância de borda, apoio, subida curta, aderência a rampas transitáveis, deslizamento em paredes e respawn. Ela ainda não é um motor geral de corpos rígidos: rampas acima do limite configurado e contatos dinâmicos não são resolvidos como física completa.

Toque em **Colisão** no HUD de jogo para ativar o diagnóstico. O body OBB real do personagem fica verde quando **grounded** e vermelho no ar. Caixas locais são azuis, esferas são violetas e envelopes de broad phase de malhas triangulares são laranja. Pontos amarelos e setas vermelhas mostram contatos e suas normais. O contador no topo indica quantos contatos foram publicados no frame. O overlay é efêmero, limitado aos 160 colisores mais próximos e não é salvo no projeto.

Áudio e eventos

`game.start` dispara semanticamente a música. Como navegadores podem bloquear autoplay, a camada web tenta novamente no primeiro gesto de teclado, HUD ou ponteiro. Salto e aterrissagem podem disparar efeitos.

Comandos úteis:

```
game status
game start
game stop
game respawn
game collision debug set {"enabled":true}
game config set {"controls":{"movementReference":"camera"}}
game config set {"character":{"stepHeight":0.35}}
game config set {"character":{"groundSnapDistance":0.3}}
game config set {"character":{"maximumSlopeDegrees":50}}
```

O estado físico é efêmero: sair do jogo restaura autoria, câmera e projeção.

21

Diagnóstico e testes

Perfil diagnóstico

Abra:

```
https://127.0.0.1:8082/apps/web/?application=diagnostics
```

No console:

```
runtime test help  
runtime test all  
runtime resources  
runtime performance
```

O perfil normal evita carregar módulos diagnósticos desnecessários.

Gates locais

Na raiz do checkout:

```
python3 tools/run_current_gates.py
```

Gates verificam arquitetura, PWA, alcançabilidade e regressões específicas. Um gate verde não substitui o teste visual de interações, PWA, animação ou jogo.

Aviso ESM no Node

Node pode emitir `MODULE_TYPELESS_PACKAGE_JSON` porque vários arquivos `.js` são ESM sem `type: module` no escopo do pacote. O runner reparsa os arquivos. A normalização foi adiada para não alterar scripts legados sem auditoria.

Solução de problemas

O build exibido não mudou

- confira `apps/web/build-info.json` na árvore servida;
- confirme que o servidor foi iniciado no checkout correto;
- veja build publicado e controlador no status;
- use **Atualizar agora**;
- feche todas as janelas e reabra;
- não publique código diferente sob o mesmo build.

Atualizar agora está desabilitado

Isso é normal quando o controlador já coincide com o build publicado. Se você alterou código sem alterar o build, o sistema não possui um identificador novo para comparar.

Tela de erro persistente

Copie o erro e a saída do console. Teste `?project=new` para separar falha do projeto demo de falha do runtime. Use **Reparar atualização** se o status PWA indicar controlador incompatível.

Objeto não seleciona

- ative a ferramenta Selecionar;
- verifique se o modo jogo ou somente cena está ativo;
- confira operação de seleção e modo objeto/componente;
- saia da edição de malha se pretende selecionar outro objeto;
- desative uma trava ou ferramenta que ainda possua o gesto.

Gizmo move a câmera

Confirme a ferramenta ativa e se o ponteiro foi capturado pela alça. Em toque, gestos de navegação e edição são arbitrados por quantidade de ponteiros; tente um gesto isolado e verifique o HUD.

Personagem penetra paredes

Confira o tamanho e orientação do proxy físico, não apenas o visual GLB. Um quadrúpede pode ser mais longo em +X que em +Z. A projeção conservadora do corpo muda com yaw.

Música não começa

Interaja com teclado, HUD ou viewer para liberar áudio. Verifique `game.audio.status` e se os assets estão disponíveis na origem.

Teste do Node não executa

No Termux:

```
pkg install nodejs-lts
```

Depois, execute novamente o teste específico. Node não é necessário para abrir o aplicativo web.

Atalhos e hábitos seguros

Atalhos comuns incluem undo/redo, duplicar, excluir, enquadrar e ferramentas de navegação/transformação. O perfil efetivo pode ser configurável; campos de texto preservam atalhos de edição próprios. Consulte a interface e [help](#) antes de depender de uma tecla numa branch diferente.

Hábitos recomendados:

- 1 confirme o build antes de testar;
- 2 salve um projeto antes de operações extensas;
- 3 use preview e cancelar quando explorar;
- 4 diferencie undo interno da malha e undo global;
- 5 rode o gate específico e depois a suíte completa;
- 6 registre dispositivo, navegador, cenário e amostras em benchmarks;
- 7 trate documentos de build antigos como histórico, não como manual atual.

O que ainda não prometer

Esta edição não afirma que o SpatialSeed já oferece:

- servidor colaborativo ou edição multiusuário remota;
- física geral com corpos rígidos, constraints e rede;
- CAD topológico completo;
- booleanas robustas para qualquer malha;
- modificadores procedurais vinculados e regeneração universal;
- compatibilidade rica completa com glTF, Collada e todos os formatos 3D;
- segurança auditada para executar plugins hostis;
- estabilidade comercial do formato de projeto.

Esses itens podem orientar a arquitetura, mas não são passos deste manual.

REFERÊNCIA

REFERÊNCIA TÉCNICA

Camadas, comandos, registros, formatos, runtime temporal, jogo, PWA, testes e limites.

1

Escopo e autoridade

Esta referência descreve as fronteiras públicas observáveis no snapshot de 18 de agosto de 2026. Ela não congela toda assinatura interna. Para listas exaustivas, consulte registros e ajuda do build carregado.

Ordem de autoridade:

- 1 código e testes da árvore;
 - 2 `apps/web/build-info.json`;
 - 3 `help, runtime test help`, registros de geometria, propriedade e ferramentas;
- 1 esta referência;
 - 2 documentos de marco.

O snapshot documentado deriva do commit [27961f1](#) e contém alterações locais de 18 de agosto validadas pelos gates locais e identificadas como build `20260818-0054mv`.

2

Modelo de maturidade

Categoria	Critério
Implementado	módulo ou fluxo presente no snapshot
Testado	coberto por gate, suíte do runtime ou roteiro confirmado
Em consolidação	contrato público existente, ainda sujeito a refinamento
Decidido	invariante registrada em docs/project/DECISIONS.md
Pretendido	direção sem implementação suficiente para uso

Contagens de testes e hashes pertencem ao build e ao Git. Não devem ser copiados como constantes em documentos vivos.

3

Camadas de estado

Região

A região representa o estado lógico aceito. Ela não conhece DOM, Three.js, gizmos ou movimentos intermediários do ponteiro.

Sandbox

O sandbox mantém uma réplica editorial, aplica comandos validados, controla revisão e histórico de undo/redo. Recuperação local preserva checkpoint e journal de comandos confirmados.

Editor

O editor mantém seleção, pivô, contexto de ferramenta, sessão de malha e previews. Esses estados não pertencem ao documento salvo.

Viewer

O viewer mantém câmera de navegação, apresentação, painéis, renderização, animação efêmera e simulação jogável. Objetos câmera persistentes são entidades do documento; ativá-los como vista é local ao viewer.

Renderer

O renderer projeta estado e overlays. Three.js, `Object3D`, materiais e lotes são recursos de projeção. Identidade lógica não pode depender do endereço de um objeto Three.js.

4

Fluxo de uma mutação

Uma mutação persistente segue, conceitualmente:

```
intenção de interface  
-> comando público  
-> validação e resolução de alvos  
-> reduzir ou serviço de domínio  
-> delta/snapshot do sandbox  
-> projeção incremental
```

Previews interrompem esse fluxo antes da publicação. Programas e experimentos produzem planos revisáveis; só o commit do plano publica comandos.

5

Comandos, queries e eventos

Comando

Solicita mudança de estado. Deve validar argumentos e autoridade antes da primeira mutação. Exemplos: `selection.translate`, `project.new`, `mesh.edit.commit`, `game.start`.

Query

Lê estado ou descreve capabilities sem mutar. Exemplos: `game.status`, `authoring.tool.describe`, `mesh.exchange.formats`, `runtime.resources`.

Evento

Relata algo ocorrido e permite reação desacoplada. Eventos de jogo como `game.start`, `character.jump` e `character.land` podem acionar áudio ou ações. Um evento não deve se transformar silenciosamente em segunda autoridade do documento.

Registro

Registros fornecem descritores de geometria, propriedade, ferramenta, ação e experimento. A interface deve consultá-los, não manter listas concorrentes.

`CommandPalette` consulta `UiActionRegistry.describe()` e `runtime.capabilities().commands`. Ações já registradas são executadas pela mesma superfície; comandos sem ação equivalente são apenas encaminhados ao Console, evitando execução implícita sem parâmetros.

6

Famílias principais de comandos

A tabela é um mapa de navegação, não uma enumeração exaustiva.

Domínio	Prefixos e exemplos
Projeto	<code>project.new</code> , <code>project.open</code> , <code>project.save</code> , <code>project.inspect</code>
Histórico	<code>history.undo</code> , <code>history.redo</code> , <code>history.status</code>
Seleção	<code>selection.*</code> , <code>selection.gesture.*</code> , <code>selection.snapshot</code>
Transformação	<code>selection.translate</code> , <code>selection.rotate</code> , <code>selection.scale</code> , <code>pivot.*</code> , <code>snap.set</code>
Criação	<code>object.create.*</code> , <code>object.placement.*</code> , <code>light.create</code>
Aparência	<code>appearance.*</code> , <code>resource.property.set</code>
Propriedades	<code>properties.describe</code> , <code>selection.properties.*</code>
Autoria	<code>authoring.tool.*</code> , <code>authoring.plane.*</code>
Contexto de edição	<code>edit.context.*</code> , <code>edit.tool.*</code> , <code>drawing.target.*</code>
Planar	<code>planar.*</code> , <code>measurement.*</code>
Caminhos	<code>path.*</code> , <code>profile.extrude.create</code> , <code>profile.revolve.create</code>
Malha	<code>mesh.edit.*</code> , <code>mesh.tool.*</code> , <code>mesh.topology.*</code> , <code>mesh.path.*</code>
Intercâmbio	<code>mesh.import.stl</code> , <code>mesh.export.stl</code> , <code>mesh.exchange.formats</code>
Animação	<code>animation.*</code> , <code>time.*</code>
Personagem	<code>character.animation.*</code>
Jogo	<code>game.*</code> , <code>game.audio.*</code> , <code>game.events.*</code>
Câmera	<code>viewer.camera.*</code> , <code>camera.object.*</code>
Viewers	<code>viewer.instance.*</code> , <code>viewer.sessions.*</code> , <code>viewer.project.*</code>
Runtime	<code>runtime.status</code> , <code>runtime.resources</code> , <code>runtime.test.*</code> , <code>runtime.performance</code>
Procedural	<code>program.plan.commit</code> , <code>procedure.plan.prepare</code> , <code>experiment.*</code>

7

Fachada de autoria

`authoring.tool.*` converge HUD, workspace, console e futuras automações sobre capacidades existentes. Ela não mantém documento ou histórico paralelos.

Operações importantes:

```
authoring.tools.list
authoring.tool.describe
authoring.tool.status
authoring.tool.activate
authoring.tool.parameters.get
authoring.tool.parameters.set
authoring.tool.parameters.reset
authoring.tool.input.bind
authoring.tool.input.use-selection
authoring.tool.execute
authoring.tool.cancel
```

Uma ferramenta declara identidade estável, contextos aceitos, parâmetros, entradas e ciclo. Adapters legados encaminham para controllers atuais.

Clipboard de propriedades

`SelectionPropertyClipboard` armazena, na sessão, valores acompanhados pelos IDs estáveis do `PropertyRegistry`. `PropertyTransferPresetCatalog` descreve presets serializáveis e aceita registros adicionais sem mudar o clipboard. Os presets padrão separam transformação segura, posição absoluta, material, textura, binding de cor e cor própria da instância.

`selection.properties.copyPreset` e `selection.properties.copy` apenas capturam valores. A query `selection.properties.clipboard.preview` resolve o destino e publica valor de origem, valor atual, compatibilidade e mudança para cada ID. `selection.properties.paste` exige a lista explícita de IDs confirmados, filtra `writable`, `editableMany` e suporte e então delega uma única mutação a `SelectionPropertyService.setSelection`.

As propriedades `appearance.color`, `appearance.colorMode`, `appearance.uniformColor`, `appearance.tint`, `instance.color` e a propriedade derivada somente leitura `appearance.effectiveColor` tornam observáveis as camadas que o HUD já compunha. Mudanças no binding entram no mesmo patch editorial e no mesmo item de undo/redo que material e transformação.

Superfícies de console correspondentes: `property presets`, `property copy`, `property clipboard`, `property paste` e `property clear`.

Documento, projeto e arquivos

Projeto .spatialseed

O arquivo transporta estado lógico e metadados do projeto. Estado transitório do viewer não deve ser serializado como parte do mundo.

Separações obrigatórias:

- documento não é seleção;
- documento não é câmera de navegação;
- documento não é preview;
- documento não é sessão física;
- arquivo não é recuperação local;
- PWA não é armazenamento do projeto.

Ordem de recuperação e lançamento do demo

`startApplication` aguarda `interfaceBinding.ready` antes de considerar o lançamento automático do demo. A política `shouldStartDefaultDemoAfterRecovery` permite o fallback em estados sem projeto persistente (`empty`, `unavailable`, `error` e `discarded`) e o bloqueia após `continued`, `restored-clean` ou `viewer-replica`. Assim, um checkpoint ou journal recuperado sempre vence o demo e reabre em autoria. O modo jogo continua local e efêmero.

Procedimentos

Bibliotecas de procedimentos usam um documento próprio, com schema `spatial-seed-procedure-library-v1`. Importar fonte não executa código.

STL

`mesh-exchange` implementa codec e serviço independentes de DOM e Three.js. `mesh-exchange-three` materializa a superfície final e aplica matriz mundial. `BrowserAssetFileGateway` realiza transporte no navegador.

STL ASCII e binário são aceitos. O descritor canônico de entrada é `buffer`. STL não preserva materiais, UV, hierarquia, animação ou unidades.

GLB/gITF de personagem

O fluxo atual usa GLB/gITF como fonte visual animada do personagem, separado do proxy físico. Isso não equivale a um importador geral rico de projeto gITF. O runtime mantém `visualYaw/facingYaw` como intenção visual e `yaw` como orientação efetivamente aceita pelo proxy OBB. A projeção da malha usa a primeira; colisão, bounds e depuração usam a segunda.

Geometria e recursos

GeometryRegistry

Geometrias entram por providers descritivos. O registry normaliza parâmetros e permite que criação, renderer, console e scripts compartilhem famílias.

Os mesmos descritores alimentam o `PropertyRegistry` sob IDs estáveis no formato `geometry.<tipo>.<parâmetro>`. O Inspector e o Console não reimplementam as regras do provider: a escrita recompõe e normaliza o descritor geométrico antes do único comando persistente. Metadados de faixa, passo, unidade, integralidade e valores enumerados permanecem no contrato descritivo.

Recursos compartilhados

Geometrias, materiais e texturas equivalentes permanecem compartilhados enquanto a semântica permitir. Identidade lógica é preservada mesmo quando a projeção usa `THREE.InstancedMesh`.

Ocorrências e hierarquia

Definições compartilhadas podem produzir ocorrências resolvidas. Transformação efetiva combina base canônica, hierarquia, animação e preview por precedência. Grupos preservam transformações locais. Expandir grupos em alvos renderizáveis é escolha explícita.

Seleção e transformação

Seleção

O snapshot de seleção contém membros e membro ativo. Operações de substituir, adicionar, remover e alternar são distintas. Gestos de área capturam geometria de tela e resolvem alvos atomicamente.

Pivô

O pivô é estado editorial. Alterá-lo não muda a transformação canônica do objeto até que uma operação use esse pivô.

Preview hierárquico

Previews são camadas transitórias. A projeção deve compor grupos aninhados e ocorrências sem gravar matrizes intermediárias. O commit produz no máximo um comando persistente por gesto.

Escala e espelho

Escala pode atravessar o pivô e produzir espelhamento. Implementações devem evitar uma matriz singular no cruzamento e preservar orientação/topologia conforme o contrato da operação.

11

Planos, esboços e caminhos

Plano ativo

`authoring.plane.*` e `adapters` de compatibilidade resolvem uma única base ativa para criação, desenho, medição e gestos de malha. A trava de câmera 2D é outro estado.

Esboço semântico

Formas 2D preservam intenção geométrica independente da malha de visualização. Perfis, caminhos e eixos são papéis explícitos de entrada.

PathReference

Referências espaciais são snapshots tipados. `Tubo`, `sweep` e `array` consomem a mesma abstração. Modificadores vinculados com regeneração automática ainda são pretendidos.

Pincel incremental

O pincel por caminho prepara objetos conforme a distância acumulada, conserva prefixo estável e confirma o lote numa operação. `u` é derivado de distância, não da quantidade final desconhecida durante o gesto.

12

Edição de malha

Sessão

`MeshEditController` mantém uma malha de trabalho e histórico interno. Entrar captura o alvo; aplicar substitui a geometria numa mutação editorial; cancelar descarta.

Topologia

O núcleo usa representação de meia-aresta transitória para conectividade e converte para descritor persistente no commit. Operações topológicas não devem editar diretamente buffers do renderer.

Álgebra de operadores

`mesh-operator-kernel` descreve domínio, componentes aceitos, invariantes, interação e categorias produzidas. A extrusão por caminho compõe segmentos da polilinha local.

Preview

`previewTopology()` parte sempre do `snapshot-base`. `commitTopologyPreview()` registra o resultado; `cancelTopologyPreview()` restaura a base. Isso evita acúmulo por `pointermove`.

Estado de consolidação

Operadores básicos estão implementados. IDs topológicos universais, atributos por canto, knife completo, booleanas robustas e backend BVH geral ainda estão em consolidação ou horizonte.

13

Runtime procedural

Linguagem afim

O parser produz AST restrita e determinística. Operadores incluem `+`, `-`, `*`, `/`, `%` e potência. Funções matemáticas são explicitamente permitidas. Não há acesso arbitrário a propriedades ou chamadas livres.

Contexto comum:

Símbolo	Significado
<code>i</code>	índice começando em 1
<code>u</code>	parâmetro normalizado
<code>count</code>	quantidade total
<code>t</code>	tempo da simulação quando fornecido
<code>dt</code>	passo temporal
<code>x, y, z</code>	posição corrente
<code>sx, sy, sz</code>	escala corrente

Worker + SES

`calc` e `program` usam um Worker com capabilities mínimas. O backend rejeita Promise, resultados não clonáveis, excesso de saída, timeout e planos acima do limite. O threat model ainda não autoriza tratar qualquer plugin hostil como seguro em produção.

Planos

Programas recebem `spatial` quando autorizados e produzem intenções. O plano é revisado e validado contra a revisão atual. Commit é atômico; discard não tem efeito.

Tempo e animação

TemporalAnimationRuntime

É a superfície recomendada para novas integrações. Consome runtime temporal, domínios e superfície de projeção. Operações podem pertencer a domínios com taxa, pausa, seek, dependências e demanda de frame.

AnimationRuntime legado

Permanece exportado para compatibilidade e regressão, marcado como obsoleto. Não deve ser escolhido por novas integrações.

Overlay

Animação efêmera compõe matriz e aparência sobre a projeção. Definição e época podem ser compartilhadas entre viewers locais; matrizes por quadro não são transportadas.

Rebase

Uma edição canônica pode exigir rebase de uma animação relativa para preservar continuidade visual. A precedência entre base, animação e preview deve ser única e testável.

15

Runtime de jogo

GameRuntime

É um orquestrador local do viewer. Controla sessão, frame demand, física, câmera, overlays, eventos e restauração. Regras específicas devem crescer em serviços ou procedimentos, não dentro de um monólito.

CharacterBodyFrame

Autoridade física do personagem. Armazena `centerOffset`, `halfExtents` e `baseYaw` em espaço local. A OBB orientada participa da narrow phase; sua AABB conservadora serve apenas à busca ampla e a contratos legados.

Visual

O visual animado não é filho escalável do proxy físico. Uma raiz transitória de pose sincroniza posição e rotação; escala visual pertence à configuração do asset. Malhas importadas emitem e recebem sombras pelo pipeline configurável do viewer. `sourceMode` aceita `default`, `custom` e `original`.

Entrada

O `snapshot` normaliza `forward`, `strafe`, `sprint`, `jump` e deltas de look. `normalizeGameDirectionalInput()` converte deslocamento radial em dois eixos contínuos, com zona morta central e intensidade preservada. Ponteiros mantêm estados separados e são combinados. A referência de movimento pode ser `camera` ou `world`.

Colisão

O `CollisionWorld` é independente de Three.js. Recebe formas numéricas:

- `local-box`;
- `sphere`;
- `triangle-mesh`.

Broad phase localiza candidatos com envelopes AABB. A narrow phase compara a OBB orientada do personagem com a forma normalizada. A câmera e as sondas de solo usam `castCollisionSegment()`, que também retorna a normal da superfície.

`CharacterPhysics` conserva um solver cinemático barato: separação por eixos para deslizamento tangencial, tentativa limitada de subida, sondas sob a base para aderência a rampas e limite angular para superfícies transitáveis. Esse contrato não introduz corpos rígidos nem estado físico persistente.

`CharacterPhysics` publica telemetria efêmera de `support`, `blocked` e `penetration`, com ID do colisor, ponto e normal. Apoio em superfície preserva a normal geométrica; bloqueios por eixo ainda usam normal axial. Essa telemetria não participa da decisão física. `GameCollisionDebugOverlay` a projeta junto das formas de colisão, limita o mundo aos colisores mais próximos e reutiliza os marcadores de contato entre frames.

Áudio e eventos

`GameEventRuntime` associa eventos a ações. `GameAudioRuntime` mantém música e efeitos substituíveis. Política de autoplay pertence à camada web.

Efemeridade

Posição física, velocidade, câmera de jogo e binding de clip não são gravados no documento. Parar libera frame demand e restaura apresentação autoral.

16

PWA e build

Fonte de build

`apps/web/build-info.json` contém `version`, `build` e `channel`. O bootstrap lê esse arquivo com `cache: no-store`.

Cadeia de cache

Imports usam query `?build=...` para separar módulos entre versões. O service worker também recebe o build na URL. `precache-manifest.json` lista recursos estáticos e hashes.

Uma promoção deve:

- 1 escolher identificador único;
- 2 atualizar manifesto e fronteiras de importação necessárias;
- 3 regenerar precache;
- 4 executar gate PWA;
- 5 testar atualização com controlador antigo;
- 6 confirmar status publicado/controlador na UI.

Estado PWA

`registerPwa()` publica:

```
supported
registered
publishedBuild
controllerBuild
activeBuild
waitingBuild
installingBuild
updatePending
scope
error
```

`updateNow()` verifica atualização, procura worker com build publicado, envia `SKIP_WAITING`, aguarda mudança do controlador e recarrega.

Viewers e coordenação local

Viewers na mesma origem usam identidade de sandbox e `BroadcastChannel`. Autoridade local publica snapshots; réplicas enviam intenções associadas a uma revisão. Entrada numa sessão aguarda o primeiro snapshot antes de participar da sucessão.

Essa arquitetura não implementa:

- transporte remoto;
- autenticação multiusuário;
- CRDT de produção;
- resolução geral de conflitos geométricos.

Perfis da aplicação

`application.default.json` define o perfil normal. O perfil diagnóstico adiciona o plugin de testes por configuração. Dependências diagnósticas não devem ser alcançáveis pela superfície de produção sem decisão explícita.

`ui.default.json` descreve composição inicial da interface. Layout persistido no navegador não cria um segundo sistema de comandos.

19

Testes e gates

Gates locais

```
python3 tools/run_current_gates.py
```

O conjunto inclui auditorias arquiteturais, PWA, alcançabilidade e regressões de marcos. O número efetivo deve ser lido da saída.

Runtime

```
runtime test help  
runtime test all
```

Suítes podem ser executadas por nome. O perfil diagnóstico é obrigatório para o plugin correspondente.

Testes específicos

Exemplo do runtime temporal legado/event-driven:

```
node tools/test_animation_runtime_event_driven.mjs
```

Qualidade visual

Mudanças de UI, PWA, câmera, gizmo, animação e jogo exigem roteiro manual. Uma regressão observada continua sendo regressão mesmo quando gates passam.

Dívida e limites conhecidos

Implementado, mas em consolidação

- runtime de jogo e personagem;
- colisão triangular sem aceleração universal;
- ferramentas por caminho e extrusão interativa;
- autoria unificada sobre adapters legados;
- visual GLB e mapeamento de clips;
- PWA em múltiplos checkouts durante desenvolvimento.

Pretendido

- cápsula opcional e contatos tangenciais completos;
- corpos dinâmicos e física geral;
- modificadores vinculados e avaliação incremental universal;
- booleanas robustas em backend substituível;
- colaboração remota com semântica de conflitos;
- segundo backend de isolamento para plugins hostis;
- gramática procedural especializada.

Regra de evolução

Uma nova implementação pode substituir backend, renderer ou índice sem alterar a identidade pública da operação. Mudanças que criam uma segunda fonte de verdade devem ser rejeitadas ou explicitamente migradas.

21

Mapa de pacotes

Área	Pacotes representativos
Estado e comandos	region-core, sandbox-core, editor-commands
Seleção e transformação	selection-operations, transform-hierarchy, math-affine
Geometria	geometry-registry, geometry-registry-three, mesh-editor-core
Autoria	edit-tools, edit-hud, spatial-references, planar-authoring
Projeção	renderer-three, viewer-rendering
Arquivos	project-io, platform-web, mesh-exchange
Programação	script-runtime, experiment-runtime
Tempo	temporal-runtime, animation-runtime
Jogo	game-runtime, character-animation, character-animation-three
Diagnóstico	runtime-test-plugin, ferramentas em tools/

Consulte [docs/project/MODULE_MIGRATION_MAP.json](#) e auditorias de arquitetura para o estado exato das fronteiras e da dívida medida.

Critérios para documentação futura

Uma capacidade pública concluída deve registrar:

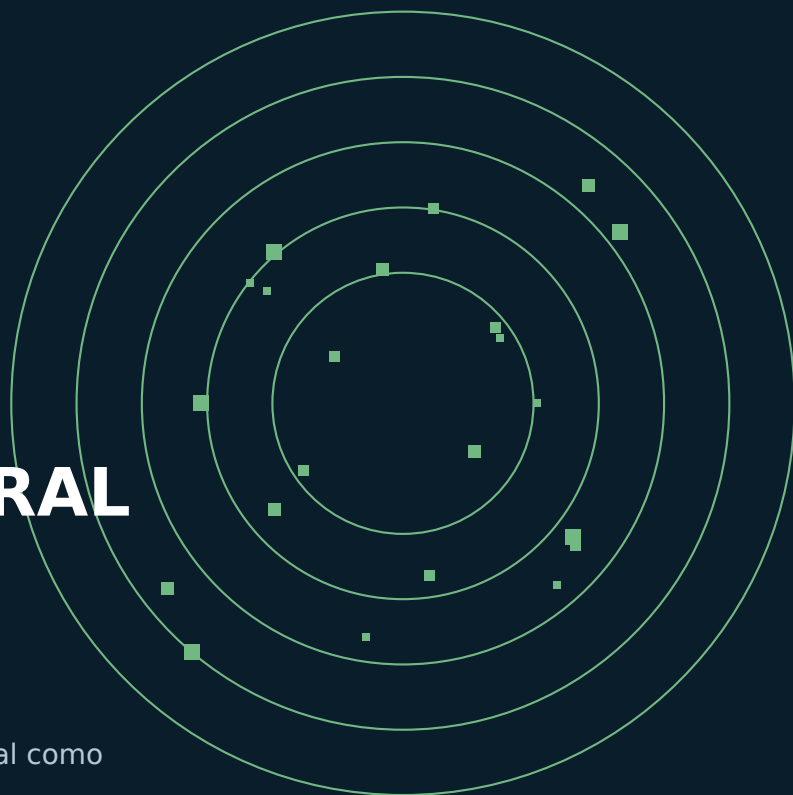
- objetivo e estado de maturidade;
- comando, query ou evento autoritativo;
- alvos e validações;
- persistência ou efemeridade;
- caminho de UI e console;
- teste automatizado;
- roteiro manual;
- limites e migração;
- relação com decisões e roadmap.

Documentos de marco permanecem históricos. Manual, Referência e Livro devem ser atualizados sem transformar planos em fatos.

ATLAS PROCEDURAL

ATLAS PROCEDURAL

Onda, hélice, roseta, cidade e Trindade Orbital como
evidência histórica da linguagem afim.



Gerar proceduralmente é escolher uma semente, um domínio, uma regra e invariantes. Não é pedir ao sistema que improvise sem contrato. Os exemplos desta parte foram criados e validados na família 0021d e permanecem como evidência histórica da linguagem afim. A sintaxe efetiva deve ser conferida por `help` no build atual.

25

Modelo matemático

Uma família procedural indexada pode ser escrita como $F(i)$, onde i é o índice inteiro da cópia. O parâmetro u normaliza o domínio: quando há mais de uma cópia, $u = (i - 1) / (\text{count} - 1)$. Fórmulas usam π para π e τ para 2π .

Três parâmetros aparecem repetidamente:

- amplitude controla o tamanho do desvio;
- frequência controla a quantidade de ciclos;
- fase desloca uma família sem alterar sua forma básica.

Em $A \cdot \sin(w \cdot u + p)$, A é amplitude, w é frequência angular e p é fase.

Incremento e posição absoluta

`move` descreve um delta. Para seguir uma curva absoluta $f(i)$, use $f(i) - f(i - 1)$. A soma telescópica reconstrói a curva e permite repetir a regra sem manter um array externo de posições.

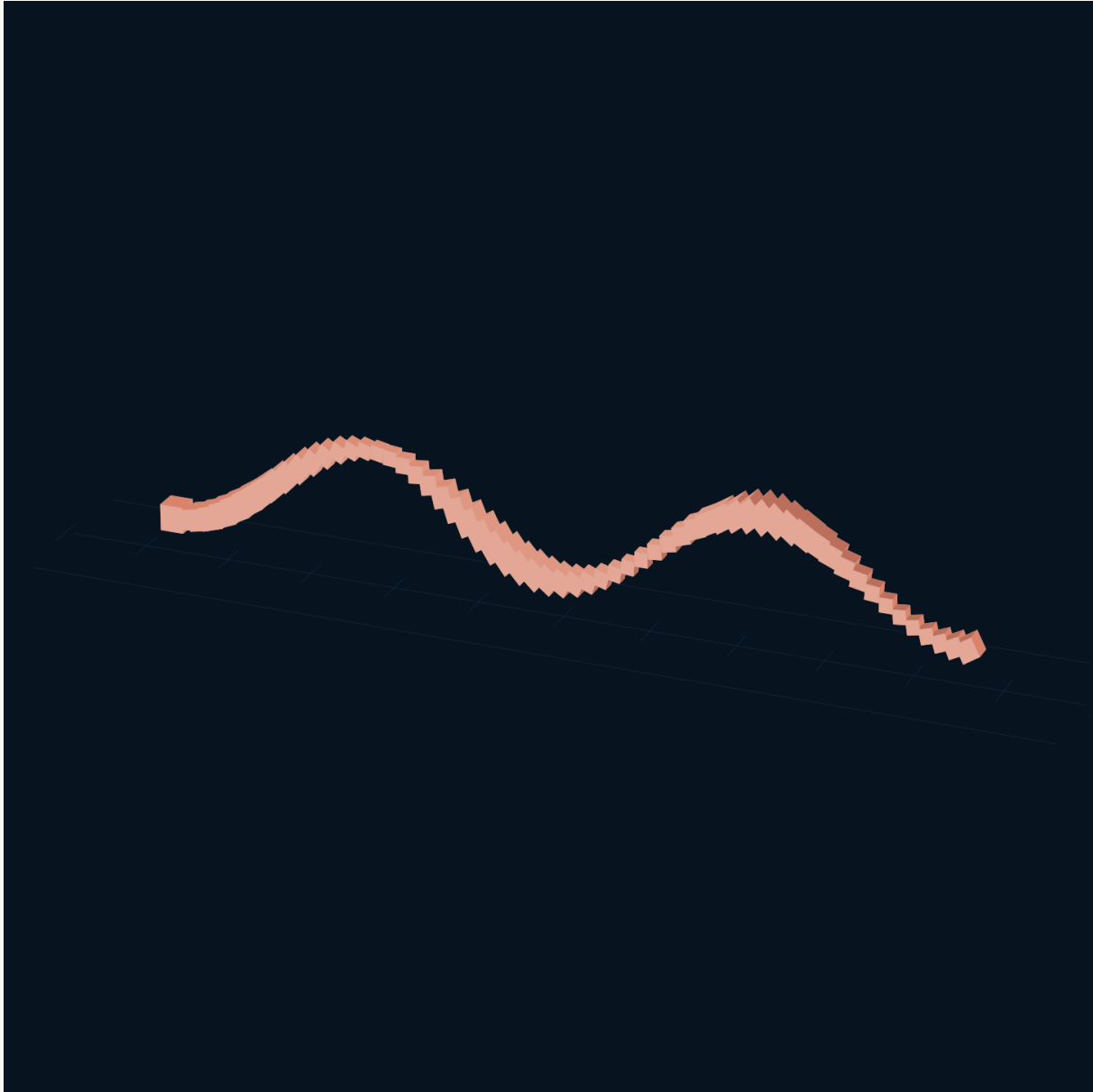
Custo e densidade

A fórmula pode descrever uma curva contínua, mas o sistema materializa amostras. Reduzir `count` é a primeira adaptação para um dispositivo móvel. Transparência, quantidade de materiais, seleção e helpers também afetam custo gráfico.

26

Onda integrada

A Onda Integrada soma um seno à altura anterior. A rotação acumula; a escala pulsa em relação à semente.



Pré-visualização editorial da Onda Integrada calculada a partir das transformações validadas.

```
select only box-2
position -12 1 0
scale 0.35 0.35 0.35
duplicate count 60 move 0.4 "0.28 * sin(2 * tau * u)" 0 rotate 0 0 5
select clear
```

O arquivo anexado preserva a listagem histórica integral.

Programa histórico integral - Onda Integrada

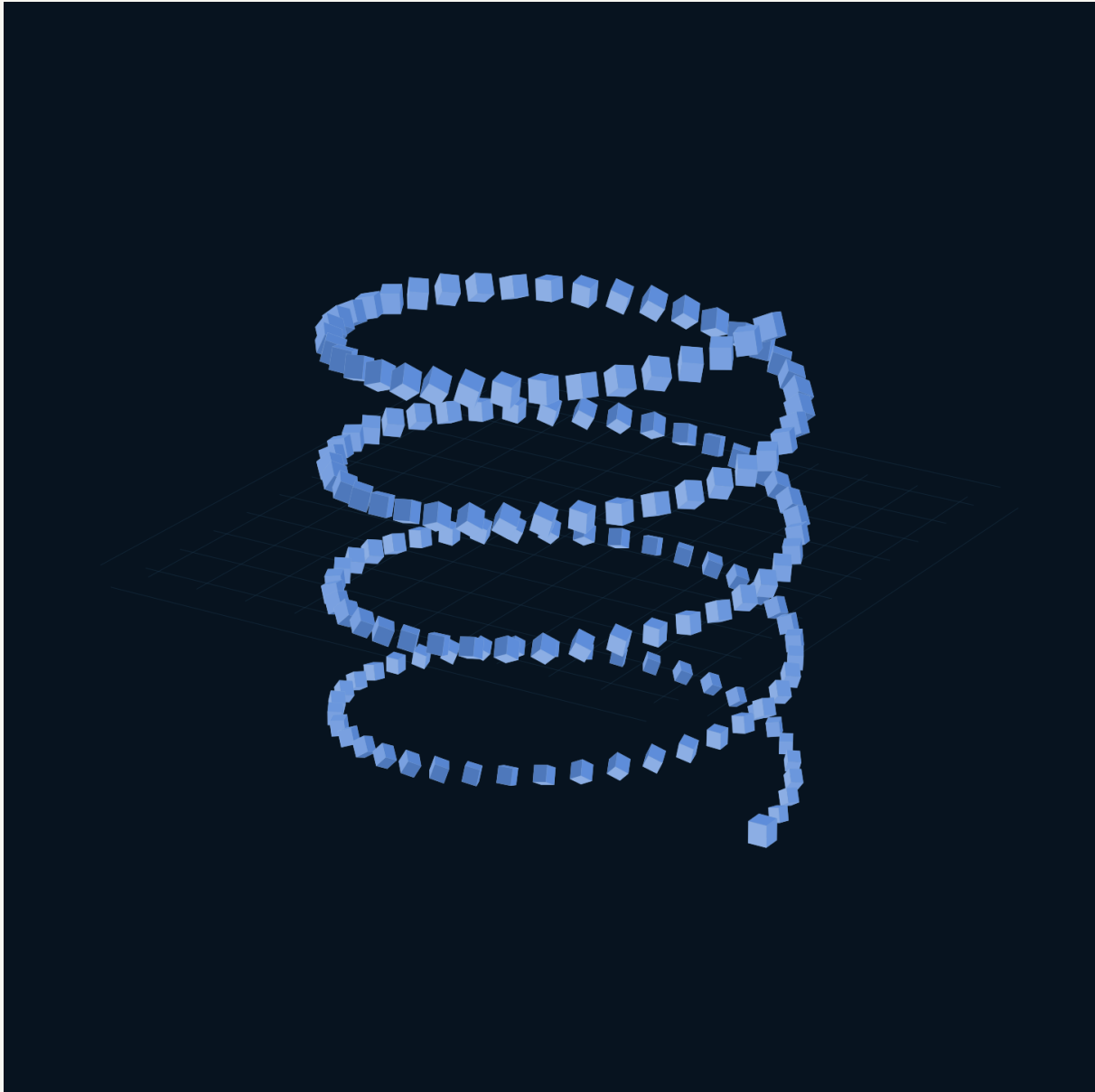
Listagem integral. As setas -> indicam apenas quebra visual da linha; para executar, use o arquivo textual anexado ao PDF.

01	select only box-2
02	position -12 1 0
03	scale 0.35 0.35 0.35
04	duplicate count 60 move 0.4 "0.28 * sin(2 * tau * u)" 0 rotate 0 0 5 scale "0.55 + 0.45 * (0.5 -> + 0.5 * sin(6 * pi * u))" "0.55 + 0.45 * (0.5 + 0.5 * sin(6 * pi * u))" "0.55 + 0.45 * -> (0.5 + 0.5 * sin(6 * pi * u))"
05	select clear

27

Hélice ascendente

Considere a curva $f(i)=(4*\cos(\tau*i/40), -4+0.05*i, 4*\sin(\tau*i/40))$. O comando fornece diferenças sucessivas de f . Quarenta passos completam uma volta; 160 cópias completam quatro voltas.



Pré-visualização da Hélice Ascendente com quatro voltas e crescimento gradual.

Variações úteis:

- aumentar a frequência angular para mais voltas;
- usar raios diferentes em X e Z para hélice elíptica;
- substituir o incremento vertical constante por expressão dependente de u ;

- aplicar fases diferentes a sementes distintas.

Programa histórico integral - Hélice Ascendente

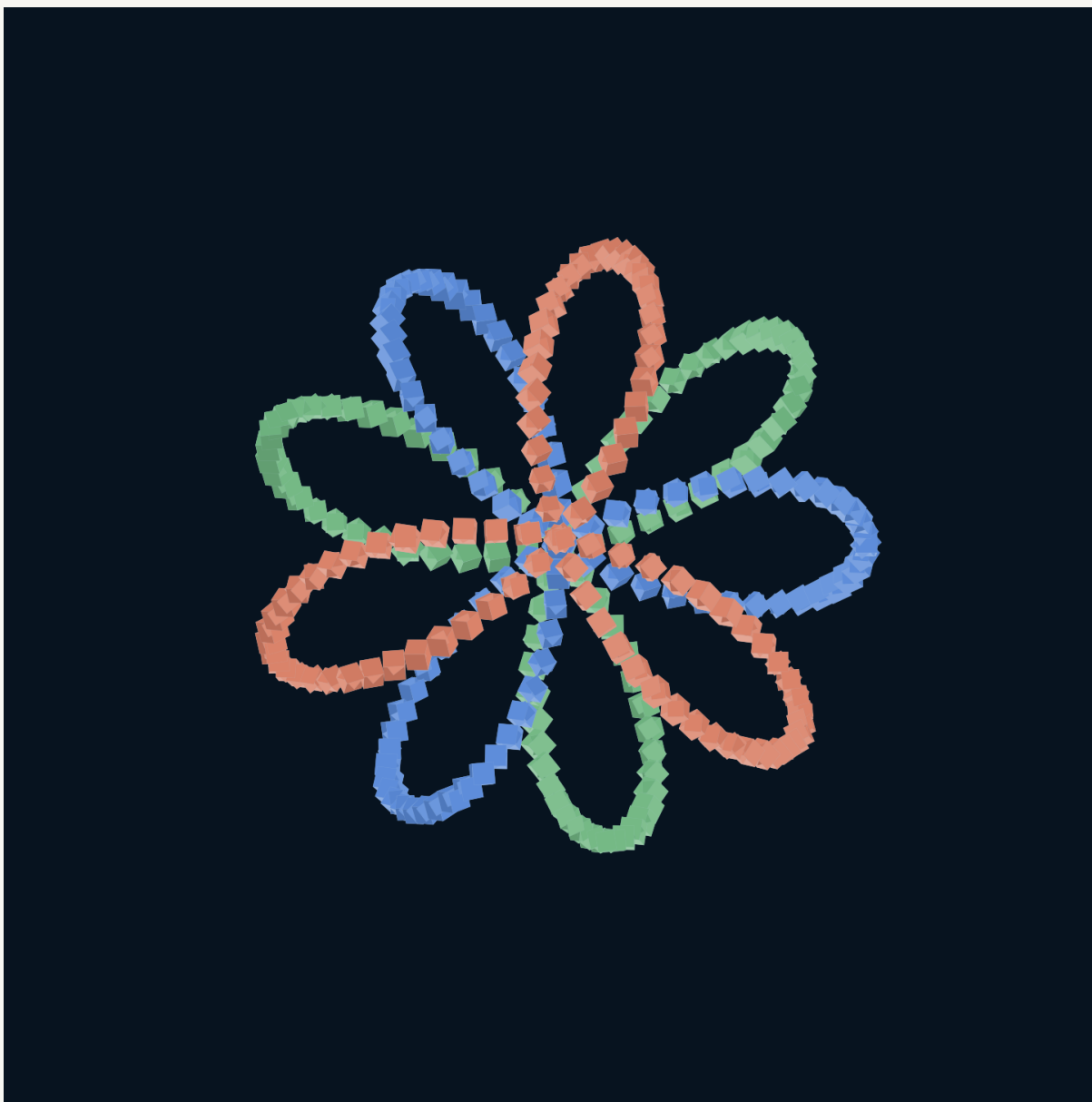
Listagem integral. As setas -> indicam apenas quebra visual da linha; para executar, use o arquivo textual anexado ao PDF.

01	select only box-1
02	position 4 -4 0
03	scale 0.18 0.18 0.18
04	duplicate count 160 move "4 * cos(tau * i / 40) - 4 * cos(tau * (i - 1) / 40)" 0.05 "4 * -> sin(tau * i / 40) - 4 * sin(tau * (i - 1) / 40)" rotate 7 11 13 scale "0.65 + 0.35 * u" -> "0.65 + 0.35 * u" "0.65 + 0.35 * u"
05	select clear

28

Roseta tricromática

A roseta usa a curva polar $r=4*\cos(3*t+p)$, com $x=r*\cos(t)$, $z=r*\sin(t)$ e pequena oscilação vertical. Três fases - 0, $\tau/3$ e $2*\tau/3$ - produzem famílias relacionadas.



Roseta Tricromática: cada cor corresponde a uma semente e a uma fase da mesma regra.

Cor permanece propriedade da semente. Alterar a paleta no Inspector antes de duplicar reutiliza a geometria sem embutir hexadecimais no programa.

Programa histórico integral - Roseta Tricromática

Listagem integral. As setas -> indicam apenas quebra visual da linha; para executar, use o arquivo textual anexado ao PDF.

```
01 | select only box-1
```

02	position 4 0 0
03	scale 0.14 0.14 0.14
04	<pre>duplicate count 180 move "4 * cos(3 * tau * i / count + 0) * cos(tau * i / count) - 4 * cos(3 -> * tau * (i - 1) / count + 0) * cos(tau * (i - 1) / count)" "0.5 * sin(6 * tau * i / -> count + 0) - 0.5 * sin(6 * tau * (i - 1) / count + 0)" "4 * cos(3 * tau * i / count + -> 0) * sin(tau * i / count) - 4 * cos(3 * tau * (i - 1) / count + 0) * sin(tau * (i - 1) -> / count)" rotate 5 7 11 scale "0.65 + 0.35 * (0.5 + 0.5 * sin(9 * tau * u + 0))" "0.65 -> + 0.35 * (0.5 + 0.5 * sin(9 * tau * u + 0))" "0.65 + 0.35 * (0.5 + 0.5 * sin(9 * tau * -> u + 0))"</pre>
05	select only box-2
06	position -2 0.4330127019 0
07	scale 0.14 0.14 0.14
08	<pre>duplicate count 180 move "4 * cos(3 * tau * i / count + tau / 3) * cos(tau * i / count) - 4 * -> cos(3 * tau * (i - 1) / count + tau / 3) * cos(tau * (i - 1) / count)" "0.5 * sin(6 * -> tau * i / count + tau / 3) - 0.5 * sin(6 * tau * (i - 1) / count + tau / 3)" "4 * cos(3 -> * tau * i / count + tau / 3) * sin(tau * i / count) - 4 * cos(3 * tau * (i - 1) / count -> + tau / 3) * sin(tau * (i - 1) / count)" rotate 5 7 11 scale "0.65 + 0.35 * (0.5 + 0.5 -> * sin(9 * tau * u + tau / 3))" "0.65 + 0.35 * (0.5 + 0.5 * sin(9 * tau * u + tau / 3))" -> "0.65 + 0.35 * (0.5 + 0.5 * sin(9 * tau * u + tau / 3))"</pre>
09	select only box-3
10	position -2 -0.4330127019 0
11	scale 0.14 0.14 0.14
12	<pre>duplicate count 180 move "4 * cos(3 * tau * i / count + 2 * tau / 3) * cos(tau * i / count) - -> 4 * cos(3 * tau * (i - 1) / count + 2 * tau / 3) * cos(tau * (i - 1) / count)" "0.5 * -> sin(6 * tau * i / count + 2 * tau / 3) - 0.5 * sin(6 * tau * (i - 1) / count + 2 * tau -> / 3)" "4 * cos(3 * tau * i / count + 2 * tau / 3) * sin(tau * i / count) - 4 * cos(3 * -> tau * (i - 1) / count + 2 * tau / 3) * sin(tau * (i - 1) / count)" rotate 5 7 11 scale -> "0.65 + 0.35 * (0.5 + 0.5 * sin(9 * tau * u + 2 * tau / 3))" "0.65 + 0.35 * (0.5 + 0.5 -> * sin(9 * tau * u + 2 * tau / 3))" "0.65 + 0.35 * (0.5 + 0.5 * sin(9 * tau * u + 2 * -> tau / 3))"</pre>
13	select clear

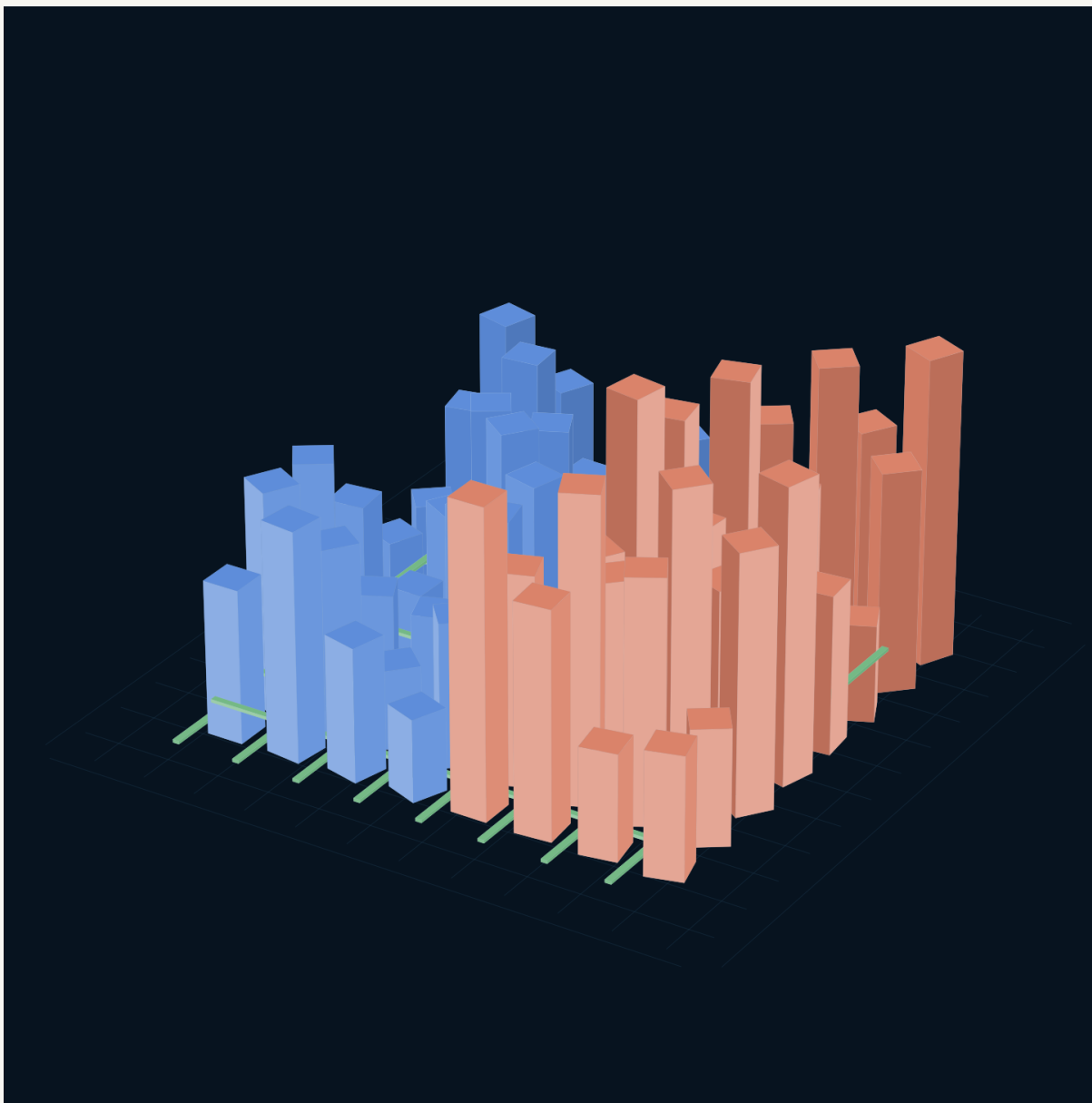
29

Cidade policêntrica

A cidade transforma um índice linear em linha e coluna:

```
col(i) = i % 4  
row(i) = floor(i / 4)
```

As diferenças entre `col(i)` e `col(i-1)` fazem o cursor retornar ao início da linha enquanto avança uma rua. Altura e centro vertical são calculados juntos para manter a base dos edifícios no chão.



Cidade Policêntrica: distritos paramétricos e eixos verdes formados por três sementes cromáticas.

Esse exemplo demonstra que a linguagem afim não se limita a curvas: grids, distritos, fachadas e infraestrutura podem ser expressos por regras pequenas.

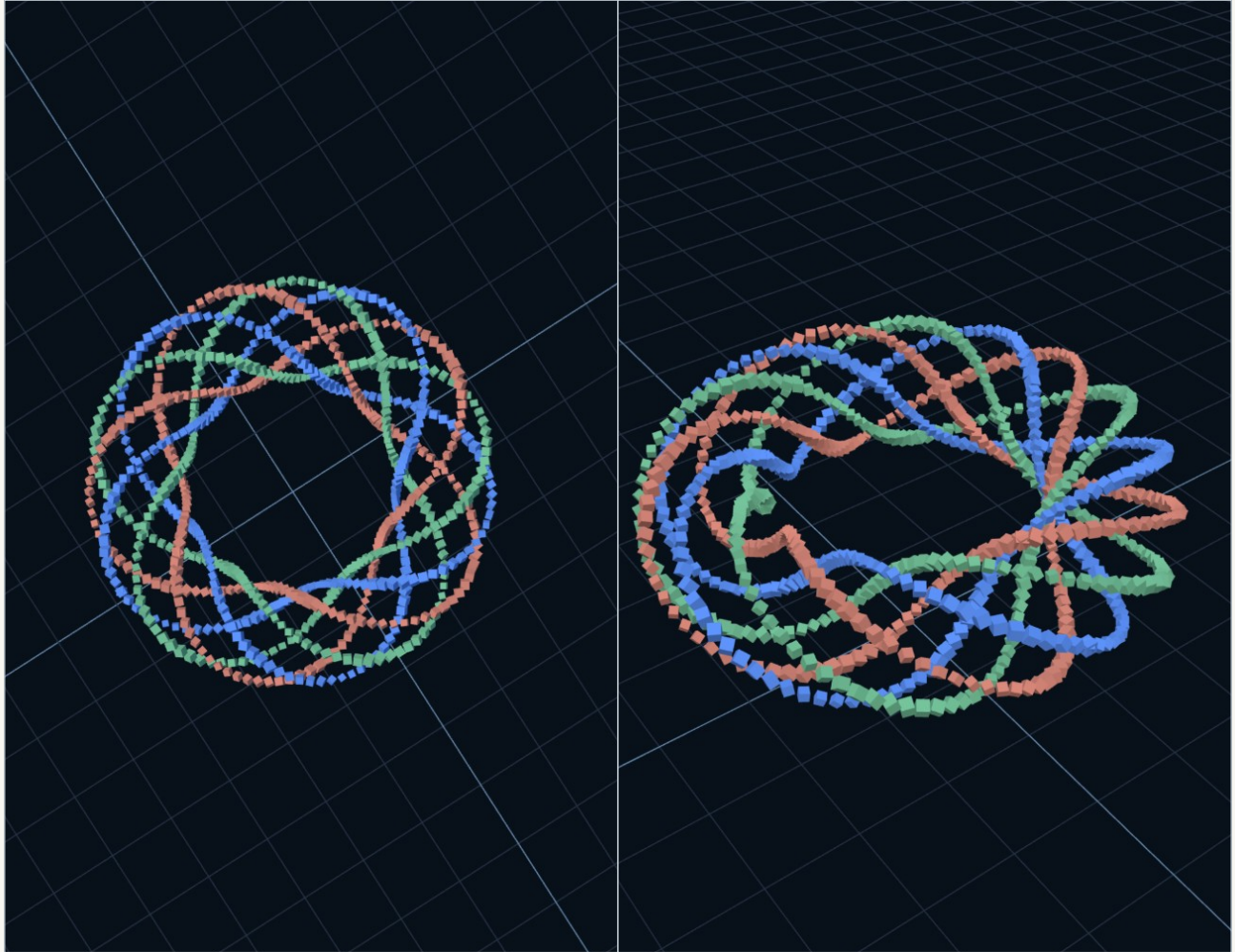
Programa histórico integral - Cidade Policêntrica

Listagem integral. As setas -> indicam apenas quebra visual da linha; para executar, use o arquivo textual anexado ao PDF.

01	select only box-3
02	position 0 0.05 -8.4
03	scale 9 0.05 0.12
04	duplicate count 7 move 0 0 2.4
05	select only box-3
06	position -8.4 0.05 0
07	rotate 0 90 0
08	duplicate count 7 move 2.4 0 0
09	select only box-1
10	position -7.2 2.5 -7.2
11	scale 0.68 2.5 0.68
12	duplicate count 31 move "2.4 * ((i % 4) - ((i - 1) % 4))" "((2.5 + 5 * (0.5 + 0.5 * sin(1.7 * -> i + 0.55 * floor(i / 4)))) - (2.5 + 5 * (0.5 + 0.5 * sin(1.7 * (i - 1) + 0.55 * -> floor((i - 1) / 4)))) / 2" "2.4 * (floor(i / 4) - floor((i - 1) / 4))" rotate 0 7 0 -> scale 1 "(2.5 + 5 * (0.5 + 0.5 * sin(1.7 * i + 0.55 * floor(i / 4)))) / 5" 1
13	select only box-2
14	position 2.4 5 -7.2
15	scale 0.68 5 0.68
16	duplicate count 31 move "2.4 * ((i % 4) - ((i - 1) % 4))" "((3 + 7 * (0.5 + 0.5 * cos(1.31 * i -> - 0.43 * floor(i / 4)))) - (3 + 7 * (0.5 + 0.5 * cos(1.31 * (i - 1) - 0.43 * floor((i - -> 1) / 4)))) / 2" "2.4 * (floor(i / 4) - floor((i - 1) / 4))" rotate 0 -5 0 scale 1 "(3 -> + 7 * (0.5 + 0.5 * cos(1.31 * i - 0.43 * floor(i / 4)))) / 10" 1
17	select clear

Trindade orbital

A Trindade Orbital combina três fios numa curva toroidal modulada. Frequências coprimas percorrem a estrutura antes do fechamento; fases distintas separam as famílias cromáticas.



Vista superior executada no SpatialSeed público.

Vista oblíqua em navegador móvel.

Se $t = \tau \cdot i / \text{count}$, raio maior $R=4$, raio menor $r=1.25$ e fase p :

```
rho = R + r*cos(5*t+p)
x = rho*cos(2*t)
y = r*sin(5*t+p)
z = rho*sin(2*t)
```

As frequências 2 e 5 são coprimas. As fases 0 , $\tau/3$ e $2\tau/3$ produzem três fios relacionados. Alterar um termo exige alterar também o termo no índice anterior para preservar a diferença telescópica.

Programa histórico integral - Trindade Orbital

Listagem integral. As setas -> indicam apenas quebra visual da linha; para executar, use o arquivo textual anexado ao PDF.

01	select only box-1
02	position 5.25 3.5 0
03	scale 0.12 0.12 0.12
04	<pre>duplicate count 240 move "(4 + 1.25 * cos(5 * tau * i / count + 0)) * cos(2 * tau * i / count) -> - (4 + 1.25 * cos(5 * tau * (i - 1) / count + 0)) * cos(2 * tau * (i - 1) / count)" -> "1.25 * sin(5 * tau * i / count + 0) - 1.25 * sin(5 * tau * (i - 1) / count + 0)" "(4 + -> 1.25 * cos(5 * tau * i / count + 0)) * sin(2 * tau * i / count) - (4 + 1.25 * cos(5 * -> tau * (i - 1) / count + 0)) * sin(2 * tau * (i - 1) / count)" rotate 7 11 13 scale -> "0.72 + 0.28 * (0.5 + 0.5 * sin(12 * tau * u + 0))" "0.72 + 0.28 * (0.5 + 0.5 * sin(12 -> * tau * u + 0))" "0.72 + 0.28 * (0.5 + 0.5 * sin(12 * tau * u + 0))"</pre>
05	select only box-2
06	position 3.375 4.5825317547 0
07	scale 0.12 0.12 0.12
08	<pre>duplicate count 240 move "(4 + 1.25 * cos(5 * tau * i / count + tau / 3)) * cos(2 * tau * i / -> count) - (4 + 1.25 * cos(5 * tau * (i - 1) / count + tau / 3)) * cos(2 * tau * (i - 1) -> / count)" "1.25 * sin(5 * tau * i / count + tau / 3) - 1.25 * sin(5 * tau * (i - 1) / -> count + tau / 3)" "(4 + 1.25 * cos(5 * tau * i / count + tau / 3)) * sin(2 * tau * i / -> count) - (4 + 1.25 * cos(5 * tau * (i - 1) / count + tau / 3)) * sin(2 * tau * (i - 1) -> / count)" rotate 7 11 13 scale "0.72 + 0.28 * (0.5 + 0.5 * sin(12 * tau * u + tau / -> 3))" "0.72 + 0.28 * (0.5 + 0.5 * sin(12 * tau * u + tau / 3))" "0.72 + 0.28 * (0.5 + -> 0.5 * sin(12 * tau * u + tau / 3))"</pre>
09	select only box-3
10	position 3.375 2.4174682453 0
11	scale 0.12 0.12 0.12
12	<pre>duplicate count 240 move "(4 + 1.25 * cos(5 * tau * i / count + 2 * tau / 3)) * cos(2 * tau * -> i / count) - (4 + 1.25 * cos(5 * tau * (i - 1) / count + 2 * tau / 3)) * cos(2 * tau * -> (i - 1) / count)" "1.25 * sin(5 * tau * i / count + 2 * tau / 3) - 1.25 * sin(5 * tau * -> (i - 1) / count + 2 * tau / 3)" "(4 + 1.25 * cos(5 * tau * i / count + 2 * tau / 3)) * -> sin(2 * tau * i / count) - (4 + 1.25 * cos(5 * tau * (i - 1) / count + 2 * tau / 3)) * -> sin(2 * tau * (i - 1) / count)" rotate 7 11 13 scale "0.72 + 0.28 * (0.5 + 0.5 * sin(12 -> * tau * u + 2 * tau / 3))" "0.72 + 0.28 * (0.5 + 0.5 * sin(12 * tau * u + 2 * tau / -> 3))" "0.72 + 0.28 * (0.5 + 0.5 * sin(12 * tau * u + 2 * tau / 3))"</pre>
13	select clear

Do atlas histórico às ferramentas atuais

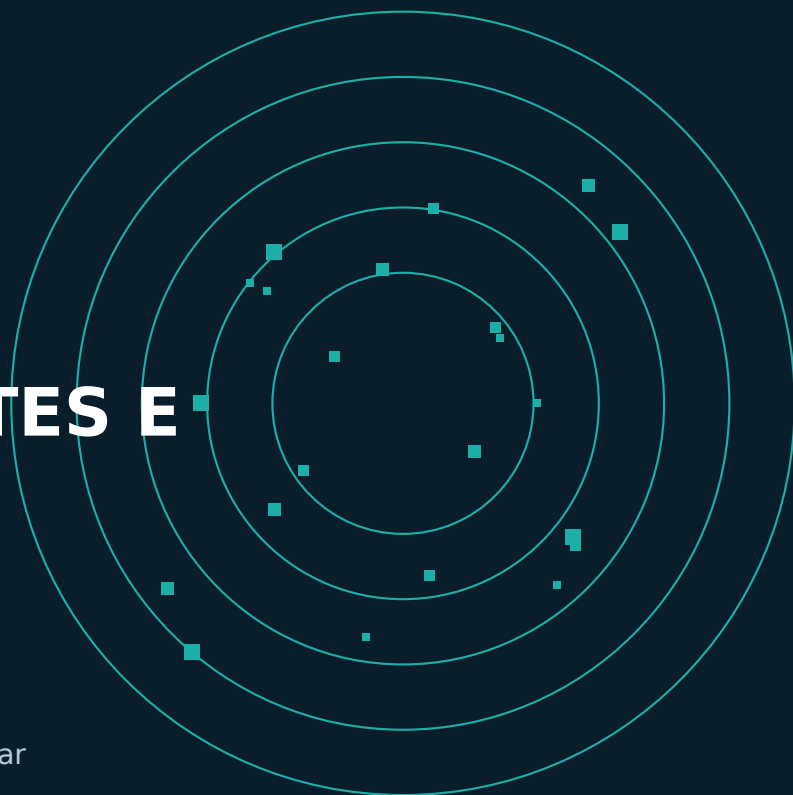
Os exemplos 0021d materializam muitos objetos por séries afins. A árvore 0054 acrescenta caminhos desenhados, distribuição incremental, formas 2D semânticas, malha editável, animação temporal e jogo. Uma evolução coerente não precisa descartar o atlas: pode transformar regras em procedimentos, assets e futuras receitas vinculadas.

O passo ainda não entregue é preservar uma receita compacta que regenere seus resultados mantendo exceções locais e identidade de componentes. Até essa fronteira amadurecer, o atlas é exemplo de linguagem e não promessa de um sistema completo de modificadores.

EVIDÊNCIA

EVIDÊNCIA, LIMITES E ROTEIRO

Como testar, medir e planejar sem transformar possibilidades em recursos prometidos.



Como validar uma versão

Uma validação útil combina:

- 1 build publicado e controlador PWA identificados;
- 2 gates locais;
- 3 `runtime test all` no perfil diagnóstico;
- 4 teste visual da área modificada;
- 5 projeto salvo e reaberto quando há persistência;
- 6 benchmark comparável quando a alegação é de desempenho.

Protocolo de benchmark

- registre commit, build, aparelho, navegador e temperatura;
- use aquecimento separado;
- colete pelo menos cinco amostras;
- reporte mediana, mínimo e máximo;
- separe lógica, projeção e frame rate;
- preserve cenário e programa junto do resultado.

Números de marcos antigos permanecem evidência histórica, não estimativa da árvore atual.

Limites assumidos

O sistema atual é um protótipo avançado, não um produto acabado. Ainda faltam estabilidade de formato, hardening de segurança, topologia robusta para casos gerais, física geral, colaboração remota e um fluxo de release simplificado.

Esses limites não anulam o que já funciona. Eles determinam onde o manual pode ser prescritivo e onde a documentação deve permanecer condicional.

34

Roteiro de maior retorno

Documentação e acessibilidade

Manter Manual, Referência e Livro sobre fontes compartilhadas; gerar ajuda a partir de registros; reduzir duplicações históricas e melhorar descoberta das capacidades já existentes.

Autoria procedural

Transformar perfis, caminhos e operações maduras em receitas vinculadas com cache e conversão explícita para malha.

Topologia

Completar IDs estáveis, atributos por canto, cortes, reparo e backend de booleanas substituível.

Jogo

Mover comportamentos específicos para eventos, procedimentos e componentes por objeto, preservando `GameRuntime` como scheduler.

Colaboração

Definir envelope causal e matriz de conflitos antes de escolher CRDT ou transporte. Convergência de mapas não substitui validade geométrica.

Motivações para continuar

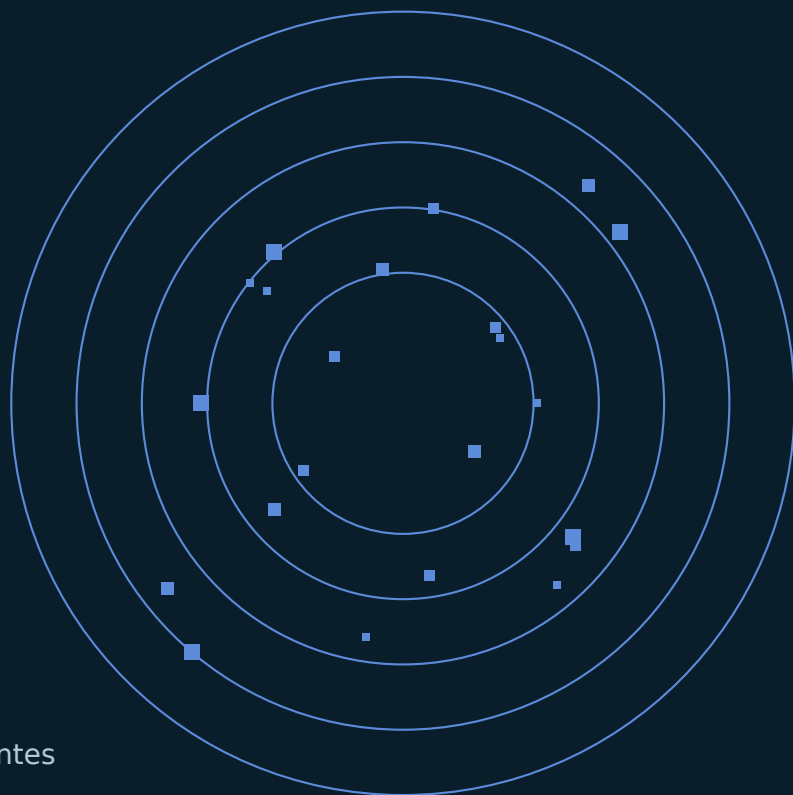
SpatialSeed reúne criação visual, semântica verificável e arquitetura para múltiplas superfícies. A mesma base permite desenhar, programar, animar e jogar sem transformar cada modo em um projeto diferente.

O sistema não precisa prever todas as formas. Precisa preservar a relação que permite que novas formas apareçam sem perder identidade, autoria e caminho de volta.

REFERÊNCIA

APÊNDICES

Entradas de ajuda, símbolos matemáticos, fontes
anexadas e nota editorial.



APÊNDICE A

Comandos de entrada

```
help  
help create  
help camera  
help tool  
procedure help  
runtime test help
```

Ajuda gerada pelo build tem precedência sobre tabelas impressas.

APÊNDICE B

Símbolos matemáticos

Grupo	Símbolos
Índice	i, index, count, u
Tempo	t, time, dt, deltaTime
Estado	x, y, z, sx, sy, sz
Constantes	pi, tau, e, phi, deg, rad, turn
Operadores	+, -, *, /, %, **
Funções	sin, cos, tan, sqrt, abs, min, max, floor, ceil, round, atan2, hypot

APÊNDICE C

Fontes anexadas

O PDF inclui Manual do Usuário, Referência Técnica, manifesto de validação histórico e cinco programas do atlas. Leitores com suporte a anexos exibem um painel de clipe.

Em sistemas com Poppler:

```
pdfdetach -list SpatialSeed_Livro_Manual_e_Atlas_Procedural_v1.0.pdf  
pdfdetach -saveall SpatialSeed_Livro_Manual_e_Atlas_Procedural_v1.0.pdf
```

APÊNDICE D

Nota editorial

A edição 1.0 abandona a arquitetura histórica do PDF como eixo do manual. O conteúdo foi reorganizado a partir do snapshot 0054: primeiro o modelo mental, depois tarefas, contratos, atlas e limites. Os documentos anteriores foram usados como fonte histórica e editorial, não como texto-base autoritativo.

Autoria principal: Rogério Duarte. Colaboração editorial e técnica: OpenAI Codex. GitHub Pages é infraestrutura de distribuição e não implica patrocínio ou endosso institucional.